基于遗传算法的黑白棋游戏人机博弈系统设计

目　录

摘要 1
Abstract 1
1　绪论 1
1.1　研究目的 1
1.2　研究意义 1
1.3　本文的工作和论文的组织结构 2
2　黑白棋平台介绍 4
2.1　黑白棋简介 4
2.2　黑白棋人机博弈的研究现状 5
2.3　黑白棋下棋经验技巧分析 7
2.3.1　行动力 7
2.3.2　前沿点 7
2.3.3　棋子数 7
2.3.4　稳定子 7
2.3.5　位置价值 8
2.3.6　奇偶性 8
3　黑白棋人机博弈问题常用算法和思想 10
3.1　黑白棋人机博弈基本思想 10
3.2　常用博弈树搜索算法 12
3.2.1　启发式搜索算法 12
3.2.2　极大极小原理 13
3.2.3　深度优先的 Alpha-Beta 搜索算法 13
3.3　基于模板估值的机器学习算法 14
4　黑白棋人机博弈引擎设计思路 15
4.1　状态的表示 15
4.2　棋盘数据结构 15
4.3　走法产生规则和游戏终止规则 17

4.4 最佳着法生成 19
4.5 估值函数的确定 20
5 遗传算法在黑白棋估值中的应用 21
5.1 遗传算法 21
5.1.1 术语说明[14] 22
5.1.2 算法基本流程 24
5.1.3 应用领域 25
5.2 遗传算法在黑白棋加权系数优化中的应用 25
5.2.1 编码与解码 26
5.2.2 适应度函数的确定 26
5.2.3 种群初始化 28
5.2.4 遗传操作：选择，交叉和变异 28
5.2.5 终止条件的确定 29
5.2.6 遗传选优总原理图和性能分析 30
5.3 加权系数优化子程序 32
5.4 加权系数优化试验结果 35
5.4.1 结果收敛情况 35
5.4.2 优化结果分析 35
5.5 采用遗传算法的黑白棋程序性能分析 35
5.6 应用到黑白棋人机博弈问题的优势和不足 36
6 系统实现、验证、分析与改进 37
6.1 系统实现关键技术 37
6.1.1 系统组成结构 37
6.1.2 人机交互界面设计 37
6.1.3 人机博弈引擎设计 39
6.1.4 人机博弈引擎调用接口 41
6.2 奇偶性的验证 43
6.3 棋力比较的传递不确定性研究 45
6.4 人机对弈和机机对弈测试 46
6.5 结果分析与比较 49
6.6 算法的进一步改进 50
6.6.1 对遗传算法终止条件的改进 50

6.6.2 使用开局库 50
6.6.3 分阶段的估值方案 50
6.6.4 动态遗传算法 51
7 结束语 52
7.1 总结 52
7.2 展望 52
参考文献 0

摘要

计算机博弈是人工智能领域最具挑战性的研究方向之一，往往涉及到了人工智能中的各种搜索算法、模式识别算法、机器学习算法、遗传算法、人工神经网络、支持向量机、贝叶斯分类等核心理论和方法，是研究人工智能的一块试金石。棋类游戏是计算机博弈问题的典型代表，半个世纪以来，世界各地已有大量学者对包括世界象棋、中国象棋、围棋、五子棋、黑白棋、跳棋在内的各种棋类游戏进行了深入的研究。1997 年 5 月国际象棋世界冠军卡斯帕罗败给“深蓝”成为机器博弈领域令世人震惊的研究成果。目前，除了围棋之外的几乎所有棋类游戏计算机的智能程度均已达到人类世界冠军的水平。

黑白棋是较中国象棋或国际象棋规则更简单的一种策略性游戏，它的状态空间更小，棋子种类更少，更适合作为初级学者将其作为研究对象来探索棋类博弈技术。目前国内外对黑白棋的博弈问题已有大量的研究，但传统的解决方法大多是基于博弈树的搜索和状态空间的结点估值，并在此基础上对搜索算法和估值方法提出了各种改进，如通过剪枝大量减少搜索的结点数、通过置换表排除不必要的重复搜索、通过设计开局库免去开局的搜索、在黑白棋的中局提高搜索的深度、通过设计更高效的估值方案提高搜索的效率等等，然而这些方法都无法从根本上解决搜索树的指数复杂度问题。当搜索深度越来越高时，计算机的计算量将会成指数级增加。在黑白棋游戏的中局，搜索结点数量和搜索深度开始逐渐增加，基于博弈树搜索的人机对弈程序的反应速度显得越来越慢，在人机对弈过程中往往会到达让人类棋手无法忍受的地步。然而，可以考虑用其他算法改善或代替搜索步骤来改变这种瓶颈状态。本文以黑白棋游戏为研究对象，在这方面做了一些初步的探索。

本文设计的黑白棋人机博弈系统分为人机界面和博弈引擎两大部分。人机界面是一个单独处理用户的输入输出的程序，负责游戏棋盘和比分的显示、下棋历史记录、悔棋等人性化的人机交互接口的处理。博弈引擎作为一个独立的模块，是对黑白棋人工智能算法的封装，对外提供一个调用接口，可供任何遵循该接口的人机界面程序调用。本文主要侧重与黑白棋博弈算法的研究和探讨，将详细介绍博弈引擎的设计思路以及验证后的性能改进方案，而人机界面程序涉及到的软件设计细节将不会占用太多篇幅。

本文设计的人机博弈引擎是基于结点估值的，基本思路是从当前结点的所有子结点出发，分别对各个游戏局面从不同的角度进行评价来获得若干个评价因子，对这些评价因子通过加权来获得该子结点上的结点估值。加权系数的好坏直接影响程序的智能程度，因而

本文引入遗传算法来优化加权系数，并针对具体问题和环境在传统的遗传算法的基础上做了一定的改进，使得程序只需要通过对当前局面上所有可以下子的位置进行估值，即可以生成最优走法，而不是像传统的基于搜索的方法那样，在当前博弈树结点上向下搜索若干层并对每个子结点估值，然后逐层向上回溯得到当前结点的估值。本文设计的方法在走法生成阶段只需要经过一个简单的线性函数的运算，免去大量递归搜索或剪枝的步骤，能大大提高算法求解速度，减少人机博弈时计算机的反应时间，且棋力达到相当的水平。

关键词： 遗传算法,人工智能,黑白棋,博弈树

Abstract

Computer Game is the most challenging research field of artificial intelligence, it often involves a variety of search algorithms and core theories and methods of artificial intelligence like pattern recognition algorithms, machine learning algorithm, genetic algorithms, artificial neural networks, support vector machines and Bayesian classification. It is the touchstone of artificial intelligence. Board games is a typical representative of the computer game problems, for half a century, there were a large number of scholars around the world have made a depth study in computer board game problems including the world's chess, Chinese chess, Go, backgammon, Othello, checkers chess game. In May 1997, the chess world champion Kasipaluo lost to "Deep Blue" and it shocked the world, which has become the most significant computer game research results in the field. Currently, the computer intelligence in almost all kinds of computer games except Go has nearly met or exceeded the level of the human world champion.

Othello is a classic strategic game which has much simple rules in comparison to Chinese chess or international chess, with smaller state space, fewer types of chess pieces, it is much more suitable for junior scholars exploring the study of the chess game technology. There are a lot of research of around the issue of Othello's game at home and abroad, but the traditional solution is based on game tree searching and node valuation in the state space, and the search algorithms and valuation method have had various improvements, such as using pruning strategy to reduce large number of nodes, by using replacement table to exclude unnecessary duplicated node searching, by designing start library to avoid searching at the beginning of the game, by increasing the depth of searching in the middle of the game, by designing more efficient valuation solution to improve search efficiency, etc., but these methods can’ t solve the exponential complexity of the searching-tree problem fundamentally. While the searching depth increases gradually, the amount of computation will be increased exponentially. In the middle of the Reversi game, the number of nodes to search and the searching depth began to increase, in a human-computer game based on the game tree searching, the reaction rate of the computer will be very slow, sometimes are intolerable by human player. However, we can consider using other methods to improve or replace the searching to change this bottleneck. In this paper, we worked on Othello game, and made some preliminary exploration in the aspect above.

This design of the Othello game system of this paper consists of two parts, the human-machine interface and the game engine. The human-machine interface is a separate program dealing with the user's input and output, its duty is to show the game board and the score for the game, and to display of chess history, undo handling, etc. The game engine, as another separate module, is the package of artificial intelligence algorithms of Othello, providing a calling interface , it can be called by any other user interface which follow the calling interface. This paper focuses on the study and research of algorithms in Reversi Game and will be focus on introducing the game engine designing ideas, as software design details involved in human-machine interface will not take up too much space.

This designing of game engine in this paper is based on the valuation of the node, generally , the basic idea is like this: starting from the current node, evaluate for every child node in different aspects on the each game situation respectively to get a number of evaluation factors, through weighted summation of the factors we can get the valuation of node, weighted coefficient impacts on the level of intelligence of the programs directly. For this case, the genetic algorithm was used to optimize the weighting coefficient, and we have do some improvements based on the traditional genetic algorithm on this specific issues, which makes the program generate the best moves by evaluating on the child nodes of the current node, rather than search down many layers from current node and valuation of each child node then gradually back up to get the valuation of current node as the traditional search-based method does. The method designed in this paper generate moves through a simple linear function, avoiding a large number of recursive search or pruning operations, it can improve the algorithm speed greatly and reduce computer's response time in a human-computer game, and the intelligence can reach a considerable level.

Keywords： GA,AI, Reversi, Othello,Game

1 绪论

1.1 研究目的

计算机博弈是一种对策性游戏，是人工智能的主要研究领域之一．它涉及人工智能中的搜索方法、推理技术和决策规划等。目前广泛研究的是确定的、二人、零和、完备信息的博弈搜索。[1]对于对弈问题，国内外传统的方法一般是采用状态空间搜索为核心进行设计，这些方法均是把博弈进行中产生的所有可能的状态看做是一颗状态空间树，而计算机下棋的过程则是搜索这棵树的子树，对当前结点的子树上的叶子结点通过计算估值得到最好的着法。估值的计算一般涉及到一个或多个评价因子，将这些因素综合起来作为评价函数来计算，因而搜索的广度和深度以及评价因子估值函数的确定是影响整个算法优劣的关键。另外，由于每次计算估值到要沿着所有可能的子树递归地向下搜索若干层，导致算法时间复杂度很高，传统的方法一般采用启发式搜索、Alpha—Beta 剪枝、着法排序、空着剪枝、哈希置换表和极大一极小原理等方法通过对部分子树进行搜索来提高效率，但这些方法都无法从根本上解决搜索树的指数复杂度问题。随着搜索深度不断加大，计算机的计算量将呈指数级急剧增加，采用上述基于搜索的各种方法设计的人机博弈程序的反应速度也随之大大减慢。

在本文的研究将遗传算法应用到黑白棋的决策规划中，实现了全新的黑白棋人机博弈引擎和完整的对弈程序以及黑白棋棋力评估、参数优化、测试程序。在传统的利用多个评价因子加权来获得估值的基础之上，引入了遗传算法，设计了一种通过遗传算法来改善加权系数的优化机制，在走法生成阶段只需要经过一个简单的线性函数的运算，免去大量递归搜索或剪枝的步骤，能大大提高算法求解速度，减少人机博弈时计算机的反应时间，且棋力达到相当的水平。

1.2 研究意义

人工智能是一门正在迅速发展的新兴的综合性很强的边缘科学，它与生物工程、空间技术一起被并列为当今三大尖端技术，它的中心任务是研究如何使计算机去做那些过去只能靠人的智力才能做的工作。[2]机器博弈是人工智能的一个重要研究分支，也是计算机模

拟人的思维能力的一个非常具有说服力的例证。黑白棋作为一个广受世界各国人民喜爱的策略性智力游戏，一直也是机器博弈研究题材之一，其状态空间结点数是一个巨大的天文数字，传统的方法大多在状态空间结点估值和博弈树搜索的基础上提出了各种改进方案以提高搜索效率，但这些方法都无法从根本上解决搜索树的指数复杂度问题，当搜索深度不断加深时，计算量呈指数级地递增，搜索效率急剧下降。然而可以考虑用其他算法免除搜索步骤来改变这种瓶颈状态。本文以黑白棋游戏为研究对象，在利用加权评价因子获得结点估计值的基础之上，设计出一种利用遗传算法来优化加权系数的的机制，并设计开发了人工智能博弈引擎，实现黑白棋游戏的人机博弈系统，通过对棋力的评估和测试验证遗传算法应用到黑白棋中的有效性。

1.3 本文的工作和论文的组织结构

本文的工作包括的内容主要有：

1) 在估值方面，讨论和分析了各种传统的下棋经验，对位置价值策略进行了优化，提出了动态位置价值策略，另外独立提出了棋势评估（亦即局面评估）方案，即以一定的加权系数对各个估值因素加权并累加；

2) 在加权系数优化方面结合遗传算法设计了一种优化方案，结合黑白棋设计出了染色体的编码与解码方法，交叉和变异算子，选择算子等，最终能够获得比较好的加权系数组合；

3) 在各个阶段策略的划分方面：开局使用开局库，在中局使用经过遗传算法优化过的系数加权来对每个可走位置估值，中局的后期（最后 10~20 层）使用遗传算法动态优化加权系数，在终局（最后 10 层）使用直接搜索，最后 10 层根据每个节点后所有的获胜次数的多少来确定走子位置；

4) 实现了黑白棋人机博弈引擎和界面，对本文方法进行了验证和评测，用实验证明了棋力比较的传递不确定性，另外验证了黑白棋的奇偶性理论。

本文的第 1 章（本章）是绪论，介绍了本文工作的目的、意义等研究背景。

第 2 章是综述，黑白棋及其人机博弈程序的发展以及黑白棋常见的下棋经验技巧。

第 3 章介绍了国内外解决黑白棋人机博弈的常用算法和思想，主要包括估值思想和博弈树的启发式搜索、极大极小原理、Alpha-Bata 剪枝算法，另外简单介绍了基于模板的机器学习方法。

第 4 章详细介绍人机博弈系统的设计思路：在黑白棋的估值中要考虑的各个因素以及如何在传统的估值的基础上利用遗传算法优化加权系数。

第 5 章介绍了遗传算法在黑白棋中的应用，首先介绍了本文要用到的遗传算法的含义、优缺点以及其发展趋势，然后具体说明了遗传算法如何运用到黑白棋的参数优化中，对遗传算法的选择交叉和变异操作、种群的初始化和遗传终止条件进行了详细的分析。

第 6 章介绍了系统的实现关键技术，系统的验证分析，最后分别从遗传算法终止条件、开局库、分阶段估值和动态地运用遗传算法四个方面提出了各种性能改进方案。

第 7 章，是对现阶段工作的一些总结和今后工作的一些设想和展望。

2 黑白棋平台介绍

2.1 黑白棋简介

黑白棋（Reversi、Othello），也叫苹果棋，翻转棋，是一个经典的策略性游戏。黑白棋是 19 世纪末英国人发明的。直到上个世纪 70 年代一个日本人将其发展，借用莎士比亚名剧奥赛罗（othello)为这个游戏重新命名。[3]黑白棋开局时，在棋盘的中央，黑白已各下有两颗子，且黑子先下，图 2-1 是黑白棋的开局界面。

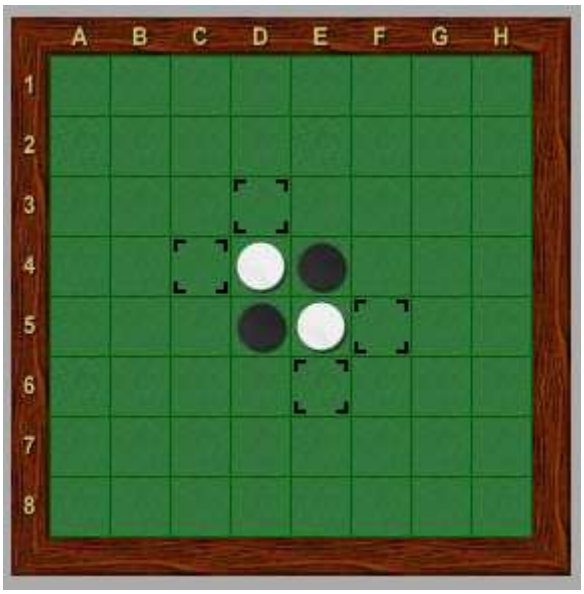


图 2-1　黑白棋的开局界面

黑白棋的棋盘是一个有 8 X 8 方格的棋盘。下棋时将棋下在空格中间，而不是像围棋一样下在交叉点上。开始时在棋盘正中有一白一黑四个棋子交叉放置，黑棋总是先下子。下子的方法是：把自己颜色的棋子放在棋盘的空格上，而当自己放下的棋子在横、竖、斜八个方向内有一个自己的棋子，则被夹在中间的全部会成为自己的棋子。并且，只有在可以翻转棋子的地方才可以下子，而且，当有棋可走时必须走棋。如果棋盘上没有地方可以下子，则该对手连下。双方都没有棋子可以下时棋局结束，棋子多的一方获胜。

黑白棋的计分方法比较简单，双方分先下偶数局数的棋(如 4 局)，胜 1 分，负 0 分，和 0.5 分，分数多的取胜。假如分数一样，就以棋子数目来计算胜负。如果双方将棋盘下满，

可以直接数棋子个数，例如，当棋局结束，黑棋有 34 个子，白棋有 30 个子，那么黑棋胜 34-30=4 个子。如果棋盘没下满，如黑棋有 34 个子，白棋有 27 个子时。有两种计分方法：日本计分法剩余的空格全部给胜方，黑棋胜 34+3-27=10 个子；欧洲的计分法剩余的空格双方各得一半，黑棋胜(34+1.5)-(27+1.5)=7 个子。

黑白棋一直深受各国人们的喜爱，在中国，日本，美国，新加坡等国家每年都会举办黑白棋公开赛。自互联网兴起，网络下棋成为了大众黑白棋对弈的主要形式。网络黑白棋比赛也日渐普及起来。网络黑白棋寻找棋友非常方便，对弈后资料保留非常妥当。中国主要的黑白棋游戏站点有 Yahoo 游戏、中国游戏网、联众游戏等。

黑白棋的游戏规则非常简单，几分钟就能学会，但要精通它却很难，这就是所谓的“学会一分钟、精通一世功”（A minute to learn, a lifetime to master）。《策略指南》（Strategy Guide）是一本是由埃马纽埃尔·拉扎德（Emmanuel Lazard）及法国黑白棋联合会（F.F.O.）创作并发行的黑白棋策略书。它的内容十分丰富，详细介绍了黑白棋中常用的一些基本策略，黑白棋初学者值得一看。《黑白棋：学会一分钟，精通一世功》（Othello: A Minute to Learn, A Lifetime to Master）由布赖恩·罗斯（Brian Rose）所著，是目前最全面的一本关于黑白棋策略的书籍。不论是新手还是高手，都会从中获益非浅。黑白棋爱好者徐暾晖将其翻译成中文，译名为黑白棋指南。[4]

黑白棋简单易学而富有趣味，不仅能增强思维能力，提高智力，而且有助于修身养性。然而，黑白棋表面上规则简单，但实际上变化复杂，是典型的易学难精游戏，它看似简单，实则奥妙无穷。

2.2 黑白棋人机博弈的研究现状

人工智能是一门正在迅速发展的新兴的综合性学科，博弈是人工智能的重要研究领域。将人工智能应用到黑白棋游戏实现黑白棋的人机博弈一直是广大黑白棋爱好者以及对此感兴趣的计算机专业人员的梦想。自从计算机人工智能兴起以来，国内外涌现出了一大批优秀的黑白棋人机博弈程序。[5]

早在 DOS 时代，就出现了有名的程序 Thor，棋力能比得上世界级棋手，但在这个时期的程序只是人工地加入行动力、位置策略、偶数重要性等有限的策略模拟人的走法下棋，策略上存在不完整性。[6]

1994 年开始，黑白棋人机博弈程序的编写方法有了突破性的进展，出现了全球黑白棋联网对战平台 IOS(GGS 的前身)，让不同的程式同时连上去互相对局，结束了以往程序员闭门造车的日子。在这个时期的 Logistello 可以说是黑白棋程序的一个里程碑，它引进了全新的模式（Pattern）评估法、快速的 MPC 搜索算法、自我学习功能等多项技术，不仅使速度大大提高，还使程序的棋力有了质的飞跃。1997 年 8 月，Logistello 以 6:0 击败世界冠军村上 健（Takeshi Murakami），从此黑白棋程序把人类棋手远远甩在后面。Logistello 是最强的黑白棋程序之一，在 1993 年~1997 年的 25 场比赛中，Logistello 有 18 场获得冠军、6 场获得亚军。在功成名就之后，Logistello 于 1998 年 1 月宣布退休，把发展空间让给其他的黑白棋程序。自从出现了具有自我学习能力的程序 Logistello 之后，机器学习被广泛的运用到黑白棋中，程序员不再把人工的策略和下棋方法等死硬地写在程序里，而是由程序自我学习，记录各种好的和坏的模式，根据实战的结果自动调整策略，例如把不同的开局棋谱根据实战的过程来评分、保存。[7]

随着计算机编程技术的发展和多线程技术的运用，现在已经有数个程序在棋力上超越了 Logistello，包括 edax, cyrano, 以及著名的 Saio。除此之外值得一提的是 Zebra（斑马），Zebra 是一款的非常优秀的开源黑白棋程序，用 C 写成，遵循 GPL 开源协议，源代码值得开发人员学习和研究。Zebra 跨 Windows，Linux，Windows CE 等多种平台。其 Windows 版本为 WZebra，除了可以下棋外，还提供了打谱、复盘、棋局分析、自我学习等功能，也可以加载 Thor 棋谱文件，进行针对性训练。在标准比赛时间(2x15 分钟)内，其搜索深度可达中局 18 至 27 步、终局 24 至 31 步。另外，日本人编的 Booby Reversi 4.0，程序很小，棋力却很强。[3]

目前国内的优秀黑白棋程序也有不少。504 Software Studio 出品的伤心黑白棋是国内棋力最强的黑白棋软件，采用模板估值技术，最高搜索深度在 16 层以上。Craft version1.2.2 是一款棋力很高的黑白棋游戏程序，自由软件，部分代码开源，提供人机对战、双人对战以及机器对战三种模式，共有 8 种不同难度的 AI 玩家供选择。下棋过程中可以通过悔棋、提示、终局求解等方式获取帮助。程序还附带了统计信息、预设棋局、自我学习、棋局分析、残局模式、音效、延迟响应、搜索过程动态显示、置换表大小设置等贴心功能。Patrick 的蜗牛黑白棋 3.0 版相比其 2.0 版棋力有了很大提升，中局估值基于行动力、前沿点、稳定子和模板，专家级的中局搜索达到 10 层，在终局搜索中改进了 WZebra 作者提供的终局算法的结构和思路，使用 hashtable 和 MTD 算法，达到了最后 24 步判断胜负，最后 20 步完全判断最后结果。Nowcan 的决战黑白棋 2.6.2 也很有实力，估值采用行动力、稳定子、

边角价值等策略，搜索极小窗口搜索(PVS)算法、置换表策略。决战黑白棋没有采用模板估值技术，程序体积小。棋力接近伤心黑白棋 3.1，但与 WZebra 相比还有一定差距。Fengart 的 Monkey 黑白棋程序带有开局库，终局搜索采用 MTD 算法，中局评价函数基于神经网络算法。此外还有李顺的雨滴黑白棋、shines 的黑白棋世界、蒋元春的亿唯黑白棋。棋力排行大概是：伤心黑白棋 v3.1 > 黑白棋世界 v1.0 > 亿唯黑白棋 v1.35 > 雨滴黑白棋 v4.02 。

2.3 黑白棋下棋经验技巧分析

2.3.1 行动力

行动力是当前走棋的一方可以选择的走子的数目。[3]计算行动力有两种方式：一种是精确计算；另一种和模板的方法类似，将各种特定棋形行动力数目事先算好，然后对于一个特定的局面，将行列斜的索引计算出来以后查表。

2.3.2 前沿点

和行动力类似。理论上，它是行动力的超集，包含了行动力。前沿点主要是用来判断一种颜色的棋子在最外延的数目。[3]前沿点越多，说明该种颜色的棋子越靠外，对方潜在可以选择的点就越多，自己越不利。

2.3.3 棋子数

棋子数就是当前黑棋或者白棋的数目。通常来说，在游戏的前面的 30 到 40 步中，自己的棋子数越少越好，越多越不利。棋子数越少，对方行动力越小。

2.3.4 稳定子

所谓的稳定子，就是无论将来对方怎么走，都不会被对方吃掉的子。[3]很显然，四个顶点上的子就是稳定子，稳定子的多少是黑白棋制胜的关键。

2.3.5 位置价值

棋盘上每一个位置的价值是不尽相同的，通常来说，顶点最佳，边次之，越靠中间越差，和顶点以及边相邻的位置最差。如果在一次着法生成中可以选择走顶点，一般都是优先走定点，当然也有特殊情况。可以走子的位置也是决定当前局面好坏的一个参数。根据经验设置了一张初始位置价值表如表 2-1 所示。

表 2-1　初始位置价值表

	A	B	C	D	E	F	G	H
1	10	-9	8	4	4	8	-9	10
2	-9	-9	-4	-3	-3	-4	-9	-9
3	8	-4	8	2	2	8	-4	8
4	4	3	2	1	1	2	3	4
5	4	3	2	1	1	2	3	4
6	8	-4	8	2	2	8	-4	8
7	-9	-9	-4	-3	-3	-4	-9	-9
8	10	-9	8	4	4	8	-9	10

在程序中，这张表是通过一个 2 维数组保存的，在游戏估值时直接查表就可以方便地得到位置价值。

随着游戏的进行，当某个边或角被某一方占领以后，其相邻位置的位置价值应该发生变化，这个问题也应在程序中予以考虑。

2.3.6 奇偶性

简单来说，奇偶性就是最后一步下子者占优势，因为最后下子方下的子因游戏的结束而不再会被对方吃掉。如果在对局中棋手都没有欠行（指出现一方无子可走而让对手下子的情形）过，那么黑棋无论何时下棋盘面都会有偶数个空位，而白棋无论何时下棋都是奇数个空位，白棋将下对局的最后一步棋，并且可以略占优势，因为他所下的及所翻转的棋子明显都将是稳定的。通常在终局算法中，奇偶性对剪枝的贡献比较大。[3]

奇偶性带给白棋一个固有的优势。然而，黑棋有办法把它转为他自己的优势：如果一位棋手欠行，奇偶性就颠倒过来；但是如果再次欠行，情况又变回它原来的状态。因此黑

棋应尽量在对局中强制进行奇数次欠行。黑棋获得奇偶性的有效办法是：迫使白棋建立一个它无法下的奇数区域。[3]

3 黑白棋人机博弈问题常用算法和思想

3.1 黑白棋人机博弈基本思想

近年来，计算机博弈一直是人工智能的重要研究分支，黑白棋作为深受大众喜爱的一种决策游戏，其规则看似简单，但其实形式变化多端，富有趣味。黑白棋人机博弈程序采用各种博弈算法使得电脑能与人类棋手对抗成为可能，计算机凭借着优秀的算法和高速的指令执行速度，在棋力上甚至能轻易超过人类棋手。

黑白棋共有 64 个空白位置，若游戏规则与围棋一样可以随便选择位置下子则状态空间为 64 的阶乘。黑方第一步下子时共有 4 个位置可供选择，以后轮流下子时双方均约有 15 个子以内的位置可供选择，到了中盘，即游戏快接近尾声的时候，双方可供选择的下子位置数随着空白位置的减少而急剧减少，游戏最少越 10 步左右结束，最多 60 步结束，可以估算，黑白棋的博弈树对应的状态空间应远小于 64 的阶乘，约在 15 的 60 次方以内，确切的说，应小于 4*2*15^46*5^10 = 98,398,379,662,661,973,287,562,933,165, 872,891,549,952, 328, 205,108,642,578,125,000，但即便如此，也不能改变这个天文数字几乎无法用目前的计算机穷尽遍历所有局面的事实。如此大的一颗树，计算机几乎不可能遍历当前结点的所有的子结点来“预测”游戏的结局画面从而选择最佳走法。

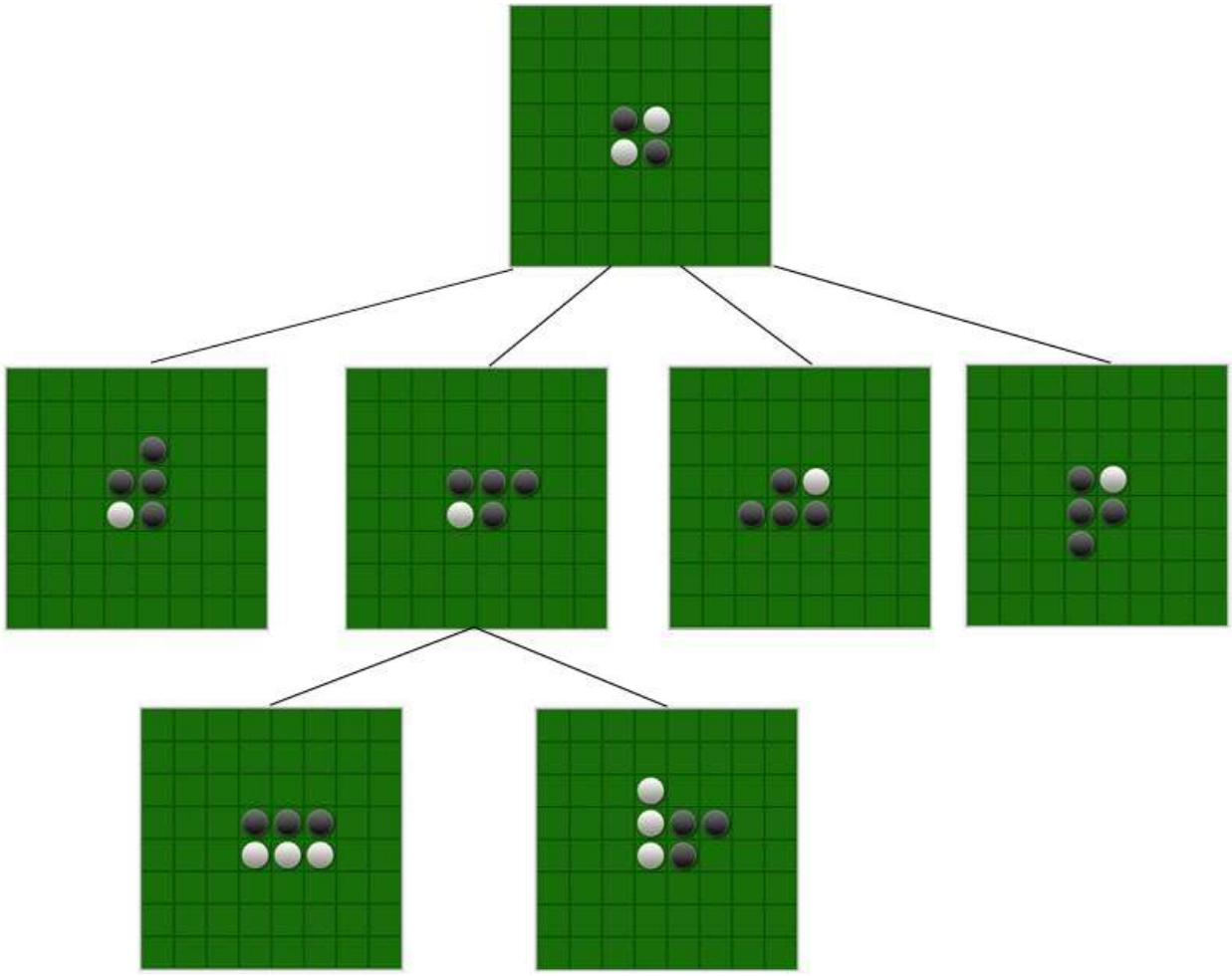


图 3-1　黑白棋前两步博弈树（第 3 层部分省去）

如图 3-1 所示，黑白棋的开局黑子先行，共有有四种可能的走法，任何一种走法后白子共有 2 种走法，图中仅列出了黑子的第二种走法后白子的两种着法。如此按在遵守游戏规则的前提下一层一层的按顺序展开，便构成了黑白棋的状态空间树，也就是博弈树。

博弈的核心思想实际上就是对博弈树节点的估值过程和对博弈树搜索过程的结合[1]。博弈程序的任务就是对博弈树进行搜索找出当前最优的一步棋。解决黑白棋的走法产生问题的一般过程都是设计一个合适的估值函数对状态空间的结点进行估值，对状态空间树进行一定深度的搜索，当搜索达到一个预定的层次时，利用一个局面评价函数对每个局面打分，然后向上回溯，计算当前局面父节点的估计值，然后继续向上回溯，一直到回到根节点，计算出根节点的估值为止。再根据极大极小原理选择一个估值最好的结点作为最佳着法结点。这种判断并不能得到绝对的最终会获胜的结点，只能在一定程度上保证该节点是最可能获胜的结点。另外，在搜索时，为了提高效率，常采用剪枝策略将那些能确定不可

能赢的结点剔除，在较低的深度剔除一些几点能大大减少搜索的结点数。最常见的剪枝策略为法有 Alpha - Beta 剪枝。[8] [9] [10]

在黑白棋求解过程中，最重要的是搜索算法，高效的搜索算法可以保证用尽量少的时间和空间损耗来达到寻找高价值的走步。但是真的想要博弈程序棋力提高，还必须有一个好的局面评价机制，即估值算法。就是说，用这个估值算法评价的局面价值必须是客观的、正确的评价局面的优劣以及优劣的程度。[11]

3.2 常用博弈树搜索算法

博弈树的状态空间搜索的一般过程是首先把问题的初始结点作为当前状态，根据一定的方法生成一组子状态，检查待搜索的目标结点是否出现在其中，若出现，则得到问题的解，若不出现，则按某种策略从已生成的状态中选择一个状态作为当前状态，重复上面的过程，一直目标状态出现或不再有合适的新当前结点和新的状态生成为止。常用的状态空间搜索算法主要分盲目搜索和启发式搜索两大类，盲目搜索主要有广度优先搜索、深度优先搜索、有界深度优先搜索、代价树的广度优先搜索和深度优先搜索，这些方法均是对问题状态空间的一种最简单的搜索，具有很大的盲目性，产生的无用结点较多，搜索空间巨大，效率非常低。启发式搜索在一定程度上能改善这个不足，主要有经典的启发式搜索算法、A*算法、与或树搜索算法等等。下面简单介绍黑白棋博弈树常用原理和搜索算法。

3.2.1 启发式搜索算法

启发式搜索[1]就是在状态空间中的搜索对每一个搜索的位置进行评估，得到最好的位置，再从这个位置进行搜索直到目标。这样可以省略大量无谓的搜索路径，提到了效率。在启发式搜索中，对位置的估价是十分重要的。采用了不同的估价可以有不同的效果。启发中的估价是用估价函数表示的，如：

$$f(n) = g(n) + h(n) \tag{3-1}$$

其中 f(n) 是节点 n 的估价函数，g(n)是在状态空间中从初始节点到 n 节点的实际代价，h(n)是从 n 到目标节点最佳路径的估计代价。在这里主要是 h(n)体现了搜索的启发信息，因为 g(n)是已知的。g(n)代表了搜索的广度的优先趋势。但是当 h(n) > g(n)时，可以省略 g(n)，

而提高效率。

3.2.2 极大极小原理

黑白棋游戏是一个典型的博弈问题，具有确定、二人、零和、全信息、非偶然的特点。[2]在博弈过程中只有相互对立的敌我二方，双方的每一步动作都是确定的，双方的利益相对。[1]假设 C1 表示我方利益，C2 表示敌方利益，则其和为 0：

$$C1 + C2 = 0 \tag{3-2}$$

1) 若我方胜利，用 $C1 > 0$ 表示，则 $C2 = -C1 < 0$
2) 若敌方胜利则 $C2 > 0$ ，$C1 = -C2 < 0$
3) 若为平局，则 $C1 = C2 = 0$

始终站在博弈一方（假设为我方）的立场上给局面进行估值，有利于我方的局面给予一个较高的价值分数，不利于我方（有利于敌方）的局面给予一个较低的价值分数，双方优劣不明显的局面给予一个中间价值分数。在博弈树的搜索过程中，从我方出发，可用如下估值函数对当前局面进行估值：

$$E(x) = C1(x)-C2(x) \tag{3-3}$$

当轮到我方下子时，E(x)的值越大代表对我方越有利，下一轮轮到对手下子是 E(x')的越小代表对敌方越有利，双方如此轮流交换下子方，当轮到我方下子时，选择 E(x)取极大值时的走法下子，当轮到对手下棋时，选择 E(x)取极小值时的走法下子，这便是极大极小算法的一般过程。这种方法对设计程序非常有利，估值函数和程序的大体逻辑不需改变，每次交换起手程序只需改变一个比较的符号即可。

3.2.3 深度优先的 Alpha-Beta 搜索算法

在极大极小搜索的过程中，存在着一定程度的数据冗余，剔除这些冗余数据，是减小搜索空间的必然做法，所采用的方法就是 1975 年 Monroe Newborn 提出的 Alpha-Beta[11]剪枝。把 Alpha-Beta 剪枝应用到极大极小搜索或者负极大值搜索中，就形成了 Alpha-Beta

搜索。

由于博弈树的“与”结点与“或”结点逐层交替出现，在生成新节点的同时，计算该结点的估值并回溯计算出其父节点的倒推值，便有可能删除一些不必要的结点。对于一个“与”结点来说，它取得当前子结点中的最小倒推值作为它倒推值的上届，称此值为 β 值；对于一个“或” 结点来说，它取得当前子结点中的最大倒推值作为它倒推值的下届，称此值为 α 值。 Alpha-Beta 剪枝的一般规律为：任何“或”结点 x 的 α 值如果不能降低其父节点的 β 值，则对结点 x 及以下的分支结点可停止搜索，并使 x 的倒推值为 β，这种剪枝称为 β 剪枝；任何“与”结点 x 的 β 值如果不能升高其父节点的 α 值，则对结点 x 及以下的分支结点可停止搜索，并使 x 的倒推值为 α，这种剪枝称为 α 剪枝。

Alpha-Beta 搜索由于过程简单、容易理解、且占用空间小等优良特点被广泛应用，成为现在主流的搜索算法的基础。但是它也有缺点，即它对于节点的排列非常敏感。对于同一层上的节点，如果排列合适，剪枝效率就很高，可以使实际搜索的博弈树达到最小树。所谓最小树就是一经向下搜索，第一个节点就是要寻找的走步。可是，如果节点的排列顺序不是从最差到最好，就有可能使剪枝一直无法进行，从而实际上搜索了整棵的博弈树。

3.3 基于模板估值的机器学习算法

黑白棋博弈问题基于搜索的解决方案尽管在搜索算法和设计思路上做出了种种改进，但仍不能从根本上解决算法的指数复杂度问题。然而，可以从估值的角度出发，通过提高估值的效率来改善整体的求解效率。出现了各种改进的估值算法，其中最成功的是模板估值，它将有着成熟理论基础的各种机器学习算法应用了进来，大大提高了估值效率。

模板也是目前黑白棋人工智能中最流行的估值参数之一，它将原先的估值参数，如行动力、位置价值等等，变成局部的估值参数，然后相加得到一个全局的估值。[12]不同的模板估值是通过事先对大量黑白棋对局利用机器学习、神经网络等方法进行各种各样的分析而得来。

4 黑白棋人机博弈引擎设计思路

黑白棋人机博弈的引擎至少需要以下几个部分：棋局的状态能够在机器中表示出来，并能让程序知道当时的博弈状态，状态之间能够方便的切换，恢复；棋盘能用合法的数据结构表示出来，使得分析棋形、吃子、推理等操作能方便的进行又能保证较高的效率；合法的走法规则如何在机器中实现，以便不让机器随便乱走而有失公平，何时应结束游戏；如何让机器从所有的合法走法中选择最佳的走法，即最佳着法生成，也是正真涉及到智能的部分。下面根据以上要点依次列出本系统的设计思路。

4.1 状态的表示

充分利用面向对象的思想，将棋盘（Board 类）的每一个格子看做为一个棋格（Grid 类）对象。则根据游戏规则，每一个棋格最多有三种可能的状态：放有黑子，放有白子，尚未下子，分别有数字 1,2,0 代表这三种情况。则 64 个棋格对应的数字按从上到下，从左到右的顺序排列形成一个 64 字符的字符串可以代表棋盘在某一时刻的状态，即某一时刻游戏的局面。游戏的初始界面上，第 4 行第 4 列和第 5 行的第 5 列各有一颗黑子，第 5 行的第 4 列和第 4 行的第 5 列各有一颗白子，此时的状态可以用字符串“0000000000000000000000000000120000002100000000000000000000000000”来表示。

4.2 棋盘数据结构

传统的棋盘表示是用二维整型数组，效率比较低，改为一维数组，能在一定程度上提高效率，但是若不使用指针将棋子连起来，在判断吃子时将变得十分麻烦。国外的黑白棋多采用性能最好位棋盘，即使用二进制来表示棋盘，然而性能总是和便利性成反比的，这种位棋盘必须通过非常复杂的逻辑位运算来处理吃子，走法产生等操作。

在本文设计的黑白棋程序中，通过 8 组 vector 容器分别存储 8 个棋格类型的指针来表示棋盘 map。这 64 个指针指向 64 个位置永远固定不变的棋格对象。每一个棋格对象内部具有 8 个棋格类型的指针分别指向上、右上、右、右下、下、左下、左、左上这 8 个方向上的相邻棋格。若棋子处在边或角上，在某个方向上已经没有相邻棋子了，则指向该方向棋格的指针为空。每一个棋格都和周围的棋格通过双向的指针互相联系，形成一个 8 叉图

的结构，对遍历棋盘和在 45 度的方向上搜索连续的棋子提供了巨大的便利，大大提高了搜索效率，另外使用标准库的容器来提供主要的基础数据结构，能充分利用了标准库通用高效的算法。图 4-1 展示的是棋盘内部的数据结构。

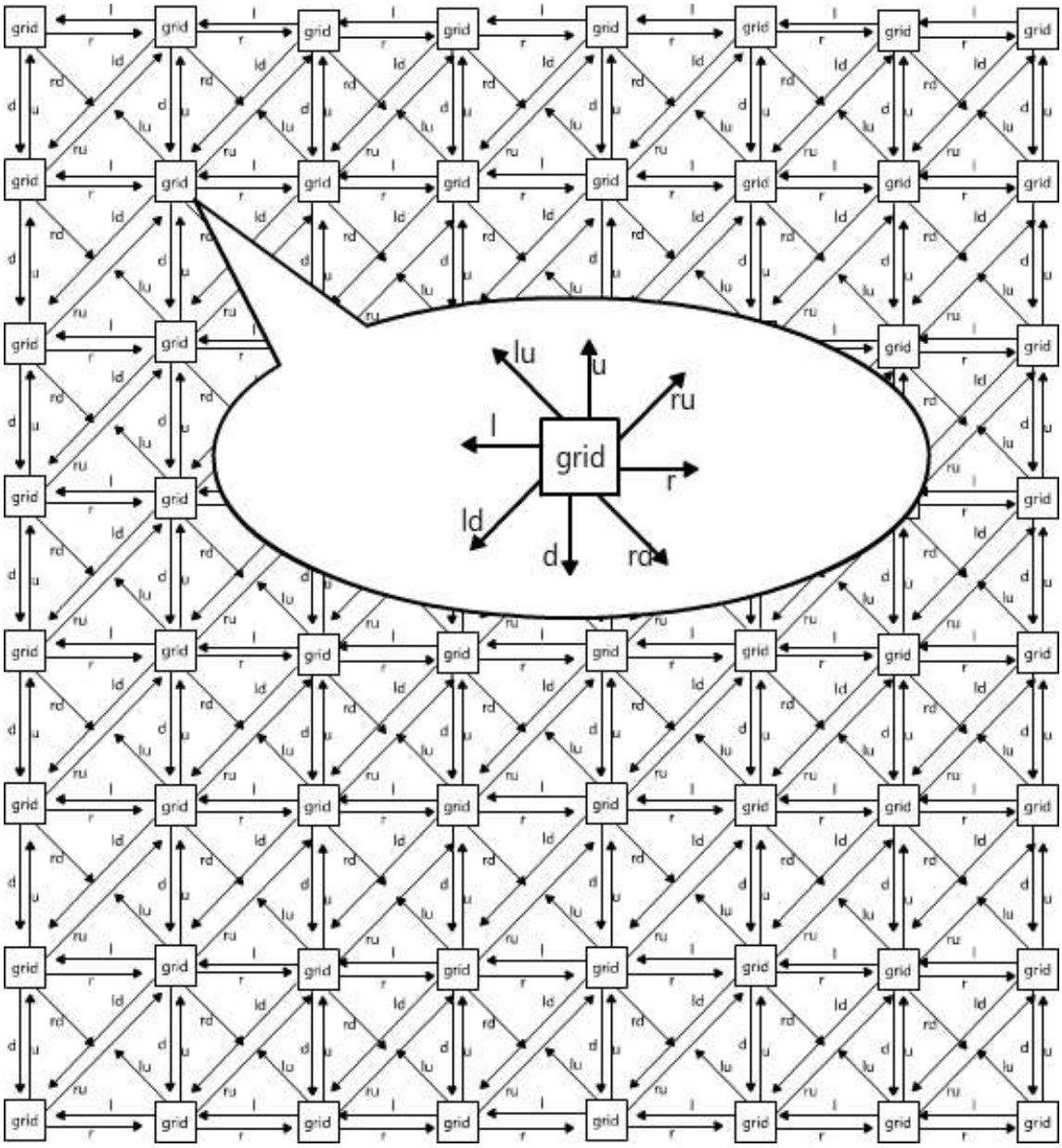


图 4-1　棋盘数据结构

若要将当前局面保存，只需要根据棋盘上每个棋格的状态生成上节描述的表示状态的字符串入栈即可，若要恢复，只需将状态栈的栈顶状态弹出，覆盖现有的棋盘状态。这种类似 8 叉树的网状数据结构与 2 维容器的结合，使得既能通过数组按横从坐标直接访问棋

盘上的每一个棋格，又能通过 8 个方向的链表以某一个棋格为基准，向任何一个方向顺次搜索、在计算位置价值、判断吃子(特别是斜对角方向上)、计算稳定子等方面非常方便，由于使用了标准库容器和链表结构，时间空间效率也比较好，然而空间效率比使用位棋盘要低很多。

4.3 走法产生规则和游戏终止规则

根据黑白棋的游戏规则，能很容易写出合法的走法产生程序。首先应当初始化游戏状态、比分、棋盘以及各个玩家的状态， 然后根据游戏规则，若棋盘上的子没有被放满且当前玩家还有子可下，则游戏尚未结束，从当前局面产生当前玩家可以走子的所有位置，当前玩家从中选择最好的位置着子后，继续重复同样的操作； 若棋盘上的子已经被放满或当前玩家已经无子可下，则游戏已经结束，输出比赛结果，并结束循环。

一局黑白棋游戏的主要流程如图 4-2 所示：

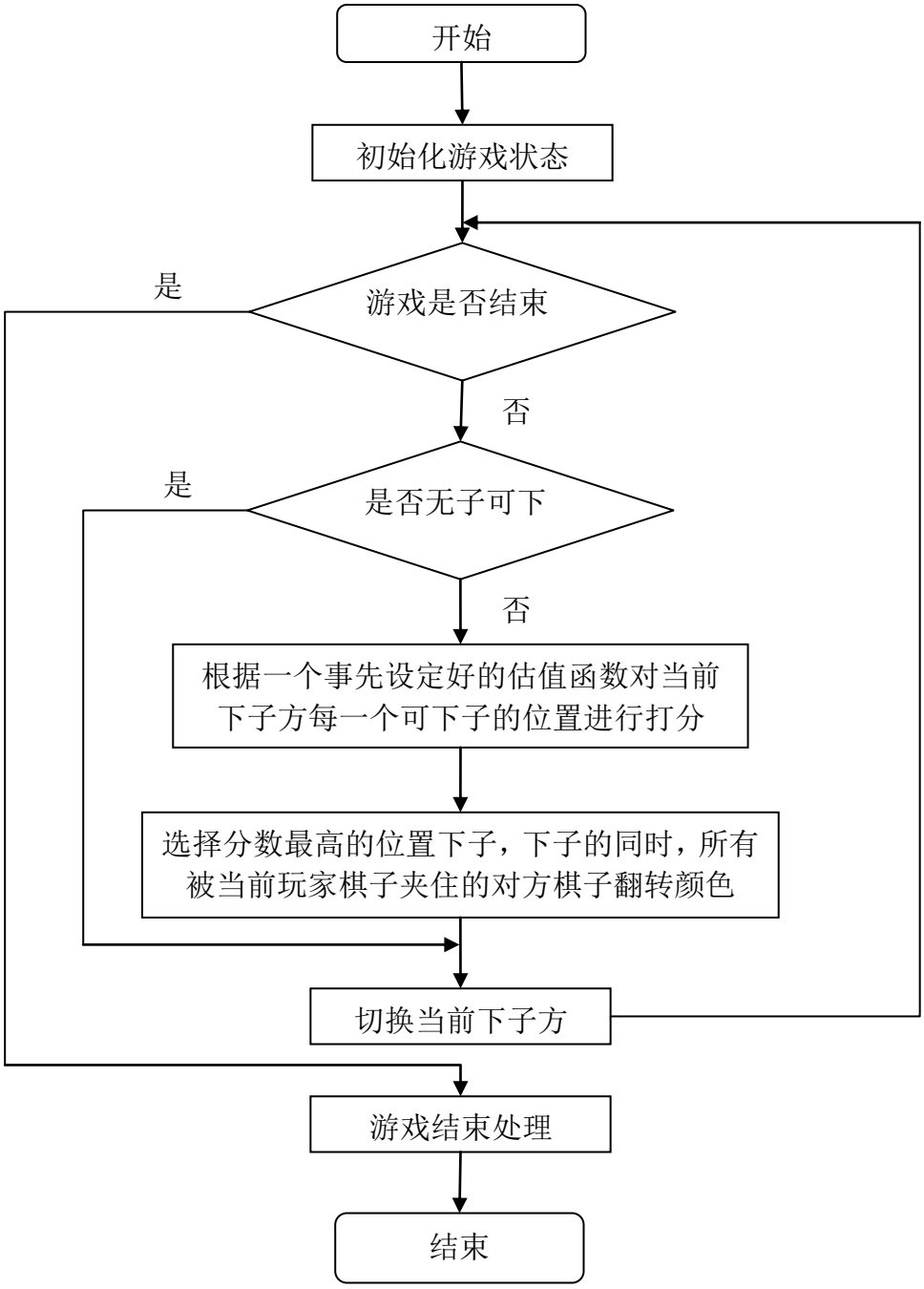


图 4-2　黑白棋游戏流程

1. 初始化棋盘，指定玩家类型以及所持棋子颜色，设置当前玩家为执黑方，置游戏状态为 PLAYING，当前步数为 1，当前回合数为 1；

2. 检查当前玩家在棋盘上的总子数，若为 0，则游游戏结束；

3. 根据游戏规则，计算出玩家所有可以下子的位置，若没有任何位置可下子则转第 7 步；

4. 若游戏状态为 ONE_PASS 则置游戏状态为 PLAYING；

5. 根据估值函数对每一个可下子的位置进行打分，选择分数最高的位置下子，下子的同时，所有被当前玩家棋子夹住的对方棋子翻转颜色；

6. 检查棋盘是否被下满子，检查方法是遍历整个棋盘，一遇到一个空白棋格就返回假，全部都不是才返回真。若下满了子，则结束游戏。

7. 若游戏状态为 ONE_PASS，说明棋局出现了连续两次双方都不能下子的僵死状态，结束游戏。若游戏状态为 PLAYING，则置游戏状态为 ONE_PASS；

8. 切换当前玩家为对手，当前步数自增 1，若当前玩家为执黑子者，则当前回合数自增 1，转第 2 步。

4.4 最佳着法生成

在游戏流程中，当前玩家在棋盘上棋子数大于零且有合法的吃子位置时，将根据一个估值函数对每一个可下子的位置进行打分，选择分数最高的位置下子。这里的估值函数的作用就是对当前位置的好坏的一个大致估计，从博弈树的角度看，估值函数用来对从当前结点产生的下一个状态进行优劣的评估，是对当前走法产生的下一个局面好坏程度的量化。估值一般是从人的角度出发根据一定的行棋经验和技巧得出来的，它们在一定程度上或多或少的影响下一个局面的形式，这些因素可以选择性地作为估值要考虑的各个分量。

除了在第 2 章介绍的常见的黑白棋下棋经验技巧中外，黑白棋还有大食策略、契入、强或前线、余裕手、边界、爬边、不平衡边、斯通纳陷阱（四通陷阱）等等一系列更加复杂的经验技巧。如果不使用模板估值，则可以在这些参数中，适当选取一些对局面评估影响更大的因素予以重点考虑，其他影响稍小的因素则可以忽略。一般可以构造以行动力为主导的估值函数，并且可选的接纳其他估值参数作为辅助参数。若使用模板估值，则模板可以成为一个唯一的估值参数。由于模板的计算不仅非常快，而且通常比较精确，可以采

用 MPC 搜索算法来进行事先的剪枝，大大提高搜索的深度和速度。目前世界上优秀的黑白棋软件大多数都是这种估值——单一模板估值。他们的搜索深度通常都在 12 层以上。[9]

在本文开发的黑白棋程序中尚未采用模板估值，使用了 6 个方面的因子来估值。分别是行动力、前沿点（潜在行动力）、稳定子、位置价值、棋势估计（吃子数减去下一次最大失子数）和奇偶性。对估值函数来说，必须在速度和估值精确度之间作一个平衡，因而不宜考虑太多。

4.5 估值函数的确定

设当前玩家所有可走子的位置集合为 avails，对与任何一个着子点 x∈avails，对在 x 处着子的好坏程度估值：

$$\text{Eval}(x) = a*\text{行动力}(x) + b*\text{位置价值}(x) + c*\text{潜在行动力}(x)$$

$$+ d*\text{稳定子}(x) + e*\text{棋势估计}(x) + f*\text{奇偶性}(x) + 1000 * \text{致死点}(x) \qquad (4\text{-}1)$$

其中系数（权重）a、b、c、d、e、f 均在区间[0,1]之间。

行动力、潜在行动力、稳定子、棋势估计、奇偶性等等均可以通过程序逻辑来做出判断，对己方有利取正值，不利取负值，越有利取值越大，越不利取值越小。位置价值则可以通过查找事先设置好的位置价值表得到。

致死点表示下这个位置将直接导致对方死棋，取值为 0 或 1，系数设置成一个大值 1000 来让估值函数绝对性地优先考虑这个因素，若这样下子能致死对方就立刻这样下子。在实际设计程序时，也可以将这个因素的考虑放入程序的逻辑中，使之与估值无关。

系数 a、b、c、d、e、f 均为在区间[0,1]之间的小数，代表了相应的因素在整个估值中所占的权重。在初始时刻可以假定它们均是同等重要的，因此可以全部取值为 0.5。在后面的遗传算法中，将对这些系数进行调整和优化。

5 遗传算法在黑白棋估值中的应用

5.1 遗传算法

以上介绍的各种算法是解决人机博弈问题的经典方法，这些理论和方法经过各种改进，实际应用效果得比较可观。然而，若在黑白棋按传统的对博弈数进行启发式的剪枝搜索并通过相对合理的估值函数来确定走法，在搜索深度不大时，效率还乐观，当将搜索深度逐渐增加以提高机器的棋力时，计算机搜索过程要处理的结点数量将呈指数级递增，计算量的大大增加导致计算机反应缓慢，时间效率急剧降低，这样反应迟钝的人机博弈软件人们是很难乐意接受的。为了克服这种弱点，必须对搜索算法从根本上做出改进，但从目前的情况来看，即使使用相当合适的评估函数和最好的剪枝和搜索策略也无法在深度急剧增大时减轻多少计算量。本文从另一个不同角度出发，利用遗传算法[2]的思想设计了一种新的下棋策略使得计算机完全不需要进行大规模的搜素便可获得最佳着子点，在介绍本方法之前首先介绍遗传算法的基本思想。

遗传算法[13]（Genetic Algorithm）是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型，是一种通过模拟自然进化过程搜索最优解的方法，它最初由美国 Michigan 大学 J.Holland 教授于 1975 年首先提出来的。遗传算法是从代表问题可能潜在的解集的一个种群（population）开始的，而一个种群则由经过基因（gene）编码的一定数目的个体 (individual)组成。每个个体实际上是染色体(chromosome)带有特征的实体。染色体作为遗传物质的主要载体，即多个基因的集合，其内部表现（即基因型）是某种基因组合，它决定了个体的形状的外部表现，如黑头发的特征是由染色体中控制这一特征的某种基因组合决定的。因此，在一开始需要实现从表现型到基因型的映射即编码工作。由于仿照基因编码的工作很复杂，往往进行简化，如二进制编码，初代种群产生之后，按照适者生存和优胜劣汰的原理，逐代（generation）演化产生出越来越好的近似解，在每一代，根据问题域中个体的适应度（fitness）大小选择（selection）个体，并借助于自然遗传学的遗传算子（genetic operators）进行组合交叉（crossover）和变异（mutation），产生出代表新的解集的种群。这个过程将导致种群像自然进化一样的后生代种群比前代更加适应于环境，末代种群中的最优个体经过解码（decoding），可以作为问题近似最优解。[14]

5.1.1 术语说明[14]

遗传算法是由进化论和遗传学机理而产生的搜索算法，在这个算法中会用到很多生物遗传学知识，下面为遗传算法中的术语说明：

染色体(Chromosome)

染色体又可以叫做基因型个体(individuals),一定数量的个体组成了群体(population),群体中个体的数量叫做群体大小。

基因(Gene)

基因是串中的元素，基因用于表示个体的特征。例如有一个串 S＝1011，则其中的 1，0，1，1 这 4 个元素分别称为基因。它们的值称为等位基因(Alletes)。

基因地点(Locus)

基因地点在算法中表示一个基因在串中的位置称为基因位置(Gene Position)，有时也简称基因位。基因位置由串的左向右计算，例如在串 S＝1101 中，0 的基因位置是 3。

基因特征值(Gene Feature)

在用串表示整数时，基因的特征值与二进制数的权一致；例如在串 S=1011 中，基因位置 3 中的 1，它的基因特征值为 2；基因位置 1 中的 1，它的基因特征值为 8。

适应度(Fitness)

各个个体对环境的适应程度叫做适应度(fitness)。为了体现染色体的适应能力，引入了对问题中的每一个染色体都能进行度量的函数，叫适应度函数. 这个函数是计算个体在群体中被使用的概率。适应度函数的设计主要满足以下条件：

A）单值、连续、非负、最大化；

b）合理、一致性；

c）计算量小；

d）通用性强。

在具体应用中，适应度函数的设计要结合求解问题本身的要求而定。适应度函数设计直接影响到遗传算法的性能。

选择(Selection)

从群体中选择优胜的个体，淘汰劣质个体的操作叫选择。选择算子有时又称为再生算子(reproduction operator)。选择的目的是把优化的个体(或解)直接遗传到下一代或通过配对交叉产生新的个体再遗传到下一代。选择操作是建立在群体中个体的适应度评估基础上的，目前常用的选择算子有以下几种：适应度比例方法、随机遍历抽样法、局部选择法、局部选择法。

交叉(Crossover)

在自然界生物进化过程中起核心作用的是生物遗传基因的重组(加上变异)。同样，遗传算法中起核心作用的是遗传操作的交叉算子。所谓交叉是指把两个父代个体的部分结构加以替换重组而生成新个体的操作。交叉算子根据交叉率将种群中的两个个体随机地交换某些基因，能够产生新的基因组合，期望将有益基因组合在一起。最常用的交叉算子为单点交叉(one-point crossover)。图 5-1 是两个染色体发生单点交叉的示例。

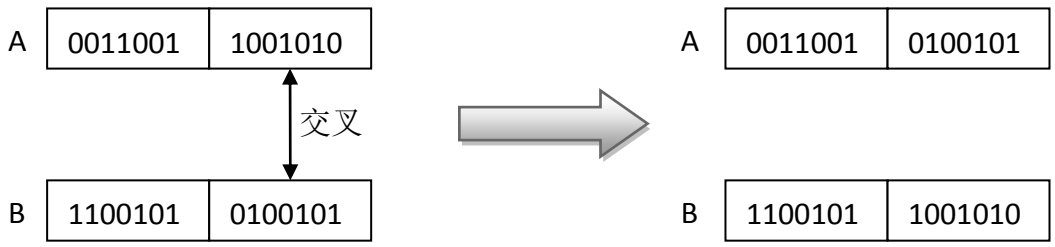


图 5-1　染色体 A,B 发生单点交叉

变异(Mutation)

变异算子的基本内容是对群体中的个体串的某些基因座上的基因值作变动。遗传算法导引入变异的目的有两个:一是使遗传算法具有局部的随机搜索能力。当遗传算法通过交叉算子已接近最优解邻域时，利用变异算子的这种局部随机搜索能力可以加速向最优解收敛。显然，此种情况下的变异概率应取较小值，否则接近最优解的积木块会因变异而遭到破坏。二是使遗传算法可维持群体多样性，以防止出现未成熟收敛现象。此时收敛概率应取较大值。

5.1.2 算法基本流程

图 5-2 为遗传算法的基本流程图：

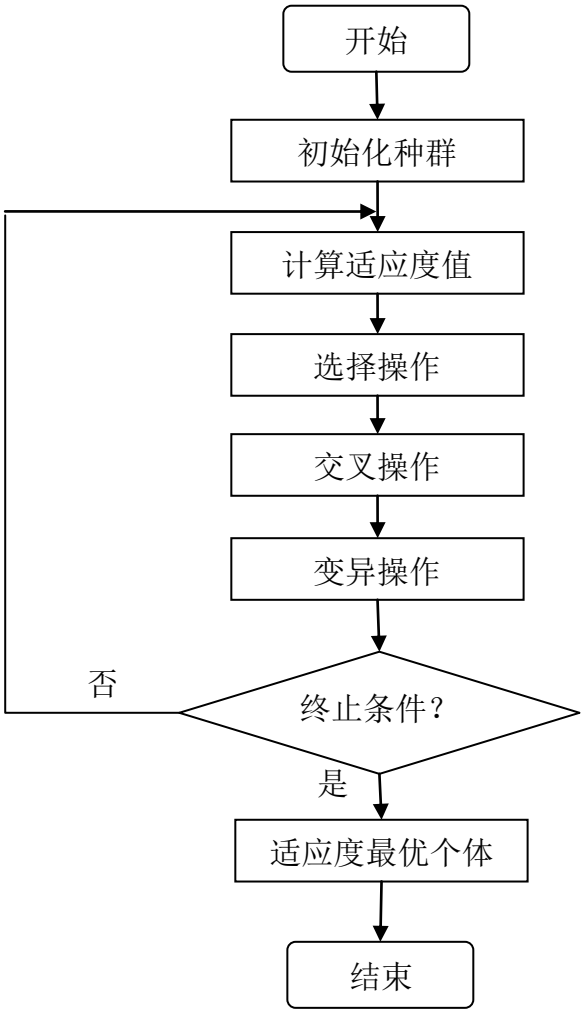


图 5-2　遗传算法流程图

5.1.3 应用领域

由于遗传算法的整体搜索策略和优化搜索方法在计算是不依赖于梯度信息或其它辅助知识，而只需要影响搜索方向的目标函数和相应的适应度函数，所以遗传算法提供了一种求解复杂系统问题的通用框架，它不依赖于问题的具体领域，对问题的种类有很强的鲁棒性，因而广泛应用于函数优化、组合优化、生产调度问题、自动控制、机器人学、图象处理、人工生命、遗传编码和机器学习等科学。[14]

5.2 遗传算法在黑白棋加权系数优化中的应用

遗传算法在在本程序中用来优化估值函数的加权系数，这也是本文的创新之处，下面介绍如具体把遗传算法应用到解决黑白棋估值函数的参数优化问题中。

5.2.1 编码与解码

将问题结构变换为位串形式编码的过程叫做编码，反之，将位串形式编码表示变换成原问题结构的过程叫做解码或译码[7]。位串形式的编码就代表了具有相应染色体的个体。

在本程序中，染色体的编码采用了二进制编码法。由于每个系数(即估值函数中的对应权值)都处在[0,1]区间之间，可将区间长度 1 分成 255 等分，每两个区间的交点加上两个端点对应 256 个不同的小数，它们组成的等差小数数列：0, 1/255, 2/255, 3/255 … 254/255, 1，256 种不同情况正好可以用一个字节表示。六个系数分别用六个字节表示，每个字节可以用 8 位二进制数表示，因而个体的染色体可以用 48 位二进制数表示，如图 5-3 所示：

01111011 11101001 01101111 01011010 10111101 10000010

图 5-3　染色体的编码表示示例

上图的基因用一组十进制数表示是 123,233,111,90,189,130，将每个数字除以 255 便得到范围在 0 到 1 之间的系数(权值)。

一组染色体实际上对应了一组估值函数系数，最终决定了程序的估值水平的好坏，也直接决定了机器智能的强弱，是程序智能水平的最终决定性因素，因而，可以把拥有一组染色体对应的估值函数的程序看做一个小机器人，它的智能程度只与相应染色体有关。

5.2.2 适应度函数的确定

需要定义一个适应度函数来评判染色体的优劣程度，来度量染色体对环境的适应能力，这是对达尔文的自然选择：适者生存，不适者被淘汰的体现。

为了获得棋力够强的小机器人对应的染色体，因而应该认为棋力越强越适应环境。

为了判断具有某个染色体的小机器人 A 的棋力强弱，最简单的方法是找一个非常强的机器人 B 如他对弈，为了消除奇偶性对结果造成的影响，至少应比赛两盘，分别是 A 先行,B 先行。最终看比赛结果，若两盘 A 都获胜了，则 A 一定比 B 更出色，若 A 两盘都输了，则

A 棋力一定不如 B，若双方各赢一盘，且 A 先行胜、A 后行负，根据后行占优势的理论，可以判断 A 稍强于 B。除此之外，暂时无法判断 A，B 棋力谁更强。

然而在初次设计的黑白棋中程序，并没有这样一个合适的够强的外部程序作为一个大自然的角色出现来筛选出好的个体（以后可以考虑使用公认棋力极强的开源程序来在这方面展开研究）。在没有外部具有较高棋力程序辅助的情况下，为了判断染色体适应度时，只能让已有个体之间互相比赛来决定强弱，为了体现公平公正的原则，需要采用循环赛制。在体育运动比赛中，循环赛是指每个队都能和其他队比赛一次或两次，最后按成绩计算名次。这种竞赛方法比较合理、客观和公平，有利于各队相互学习和交流经验。

具有 N 个个体的种群举办一场循环赛共需比赛场数：

$$\text{总场数}(N) = N(N-1) \tag{5-1}$$

在开始确定适应度函数之前，必须提出使用估值和遗传算法带来的两个问题：棋力比较的传递不确定性和遗传算法的不精确性。

棋力比较的传递不确定性

若有三个不同棋力的基因 A，B，C，已知对应机器人的棋力 A>B 且 B>C，那么 A 是不一定大于 C 的。本文第 6 章中，在完成系统设计后，这个结论得到了实验的证明。

遗传算法的不精确性

遗传算法不能得到全局最优解，只能得到在一定程度上比较优秀的局部最优解。由于无法判断什么时候得到了较优解，只能粗略地获得整体水平较优的种群并从中取出在一定概率上较优的解，因而使用遗传算法具有不精确性。

在明确了上述两个观点的前提下，初步设计了一种判断个体优劣的机制如下：

在一个种群规模为 N 的具有不同染色体的个体之间举行循环赛，让每个个体都有机会和任何其他 N-1 个个体进行两场比赛（执黑执白各一场），分别记录它们各自获胜(win)，战败(lose)，平局(draw)的总次数，最后计算出每个个体的总分和胜率，其中总分为获胜次数减去战败次数，胜率为获胜总次数除以该个体的比赛总次数(2N-2)。总分即为个体的适

应度值。适应度函数就是个体染色体 c 到他对应的机器人在所属种群循环赛中的总分之间的映射：

$$适应度(c) = win - lose \tag{5-2}$$

5.2.3 种群初始化

为了获得对于群规模为 N 为的一组个体，可以首先随机产生一个染色体，在其基础上连续发生大概率的基因突变，获得 N-1 个新的基因。也可以直接随机产生 N 个完全不同的染色体。最终形成第一代种群。

5.2.4 遗传操作：选择，交叉和变异

在某一代个体之间举行完循环赛之后，根据总分排序，从中选出分数最高的第一名并保留其染色体（若存在并列第一名则取最先出现的那一个），让其他的 N-1 个个体之间先按照一定交叉概率随机地进行杂交，即染色体发两点生交叉，然后让每一个个体的染色体发生一定突变概率的基因突变，最后形成 N-1 个具有新染色体的个体。它们和之前被保留的第一名融为一体再次形成具有 N 个个体的种群，即新的子代。

也可以每次保留前二名（或前三名）来减少优秀基因的流失，从而提高收敛速度和遗传效率。但应该在上一代记住前二名（或前三名） 之间的比赛结果而在下一代中免去重复比赛带来的开销。

图 5-4 为 y 基因内部发生基于突变和 x,y 基于之间发生基于重组的试验结果：

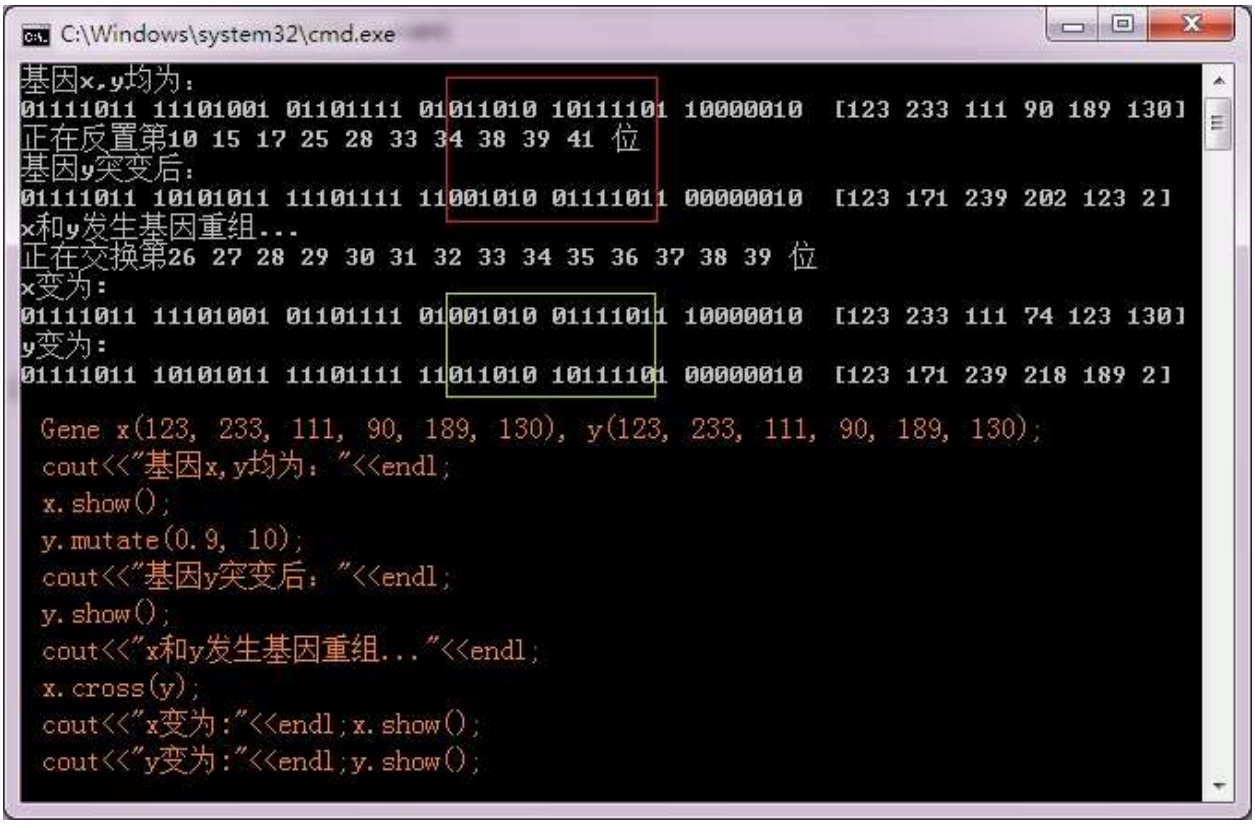


图 5-4　基因突变和基因重组试验结果

5.2.5 终止条件的确定

重复上述比赛计分，选择，遗传和变异产生新的子代的过程，直到某一代种群的第一名和最后一名总分之差小于最小差别阈值为止。

终止条件是 A:“直到某一代种群的第一名和最后一名总分只差小于最小差别阈值为止”，而不是 B:“直到某一代种群的第一名的总分达到指定阈值或胜率超过指定阈值”的原因是，B 表示的是最终获得了了一个在该代种群的各个个体之间棋力差异达到非常高的程度的个体，但由于上述不精确性原理，在基因突变和染色体交叉的过程中，可能会偶然的出现下个子代个别个体棋力较强，其他整体较差的特殊情况，这时的第一名虽然能打败其他 N-1 个较差的个体，但不一能说明它一定是所有代中棋力在一定概率下最强的，也许它仍不能打败上一代的第二名。 而 A 强调的是第一名和最后一名的棋力相当时，结束上述循环，虽然前面有不精确理论，但可以近似认为第一名是最好的，最后一名是最差的，当最好的和最差的棋力相当的时候，说明当前代种群的所有个体棋力强弱情况分布更平均，由于总是保留上一代近似最强的染色体到下一代，因而可以认为整个种群整体水平在遗传和进化

的过程中是在逐渐变强的，在整体水平最强且分布均匀的个体之间，随意选择一个个体其真实棋力确实达到了一定程度的概率更大，一般认为前三名中的任何一个可以作为遗传选优的最终结果。

5.2.6 遗传选优总原理图和性能分析

遗传算法的这一系列操作的最终目的就是获得较优染色体。图 5-5 为总原理图：

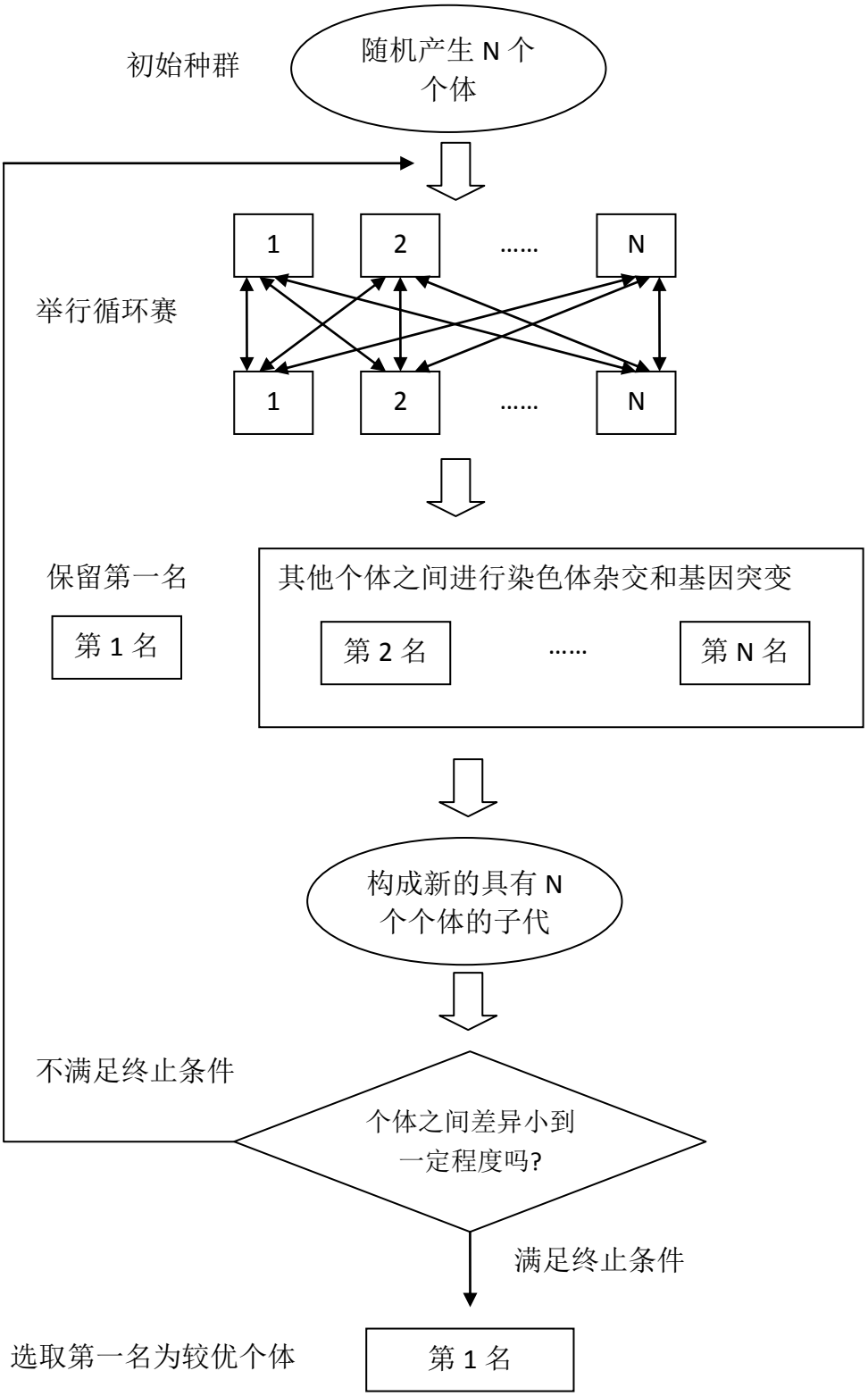


图 5-5　通过遗传算法确定较优染色体原理图

算法的主要开销在比赛上，每一个子代的产生需要经过 n(n-1)场比赛，每场比赛最多需要进行 60 步轮流下子，每一步下子都要对平均余约 10 个下子点去搜索是否满足各个估值因素并加权得到估值。若开局采用开局库，终局 10 步之前通过硬搜索直接判断输赢，则可以减少越 20 步走子时估值的计算，还剩 40 步，每场比赛的约 400 次估值的计算，可以认为评价每场比赛估值的计算量的平均值为 A。对于含有 n 个个体的种群来说，采用上述方法获得一个子代的总计算量为：A*n(n-1)。假设遗传 K 代才能收敛，则总计算量为：K*A*n(n-1)。算法的时间复杂度为 $O(n^2)$。

5.3 加权系数优化子程序

为了得到较好的加权系数，按照上面的思想，设计出了加权系数优化子程序，界面如图 5-6 所示：

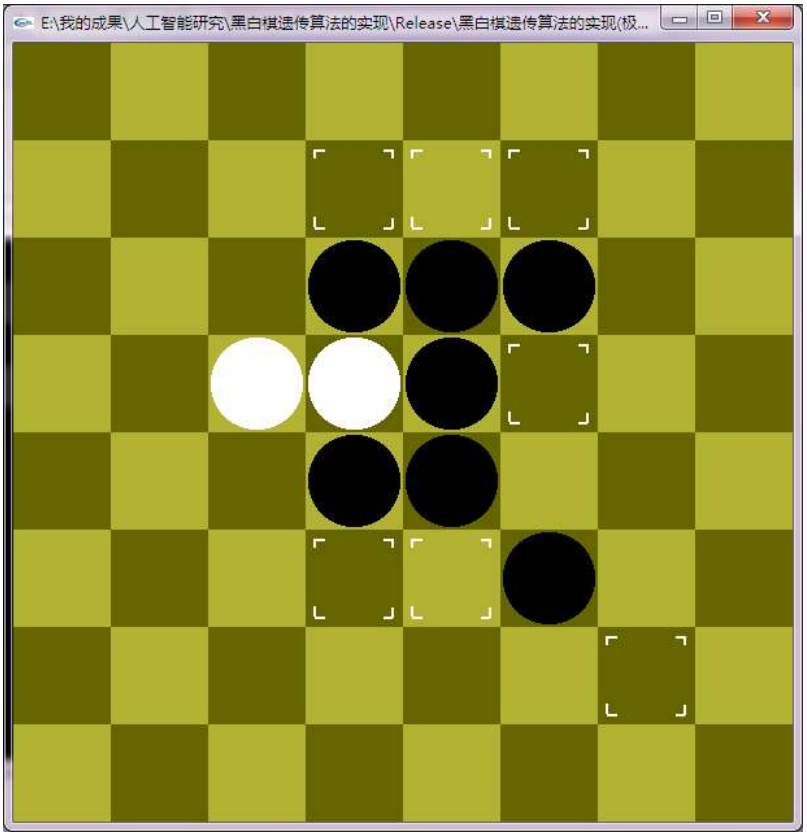


图 5-6　加权系数优化子程序界面

对于一个小规模种群（规模为 10）的种群进行初步的遗传测试，遗传 69 代便得到了

胜率在 90%以上的染色体，图 5-7 为开始遗传时的数据，图 5-8 为遗传过程中，图 5-9 为算法终止时的结果。

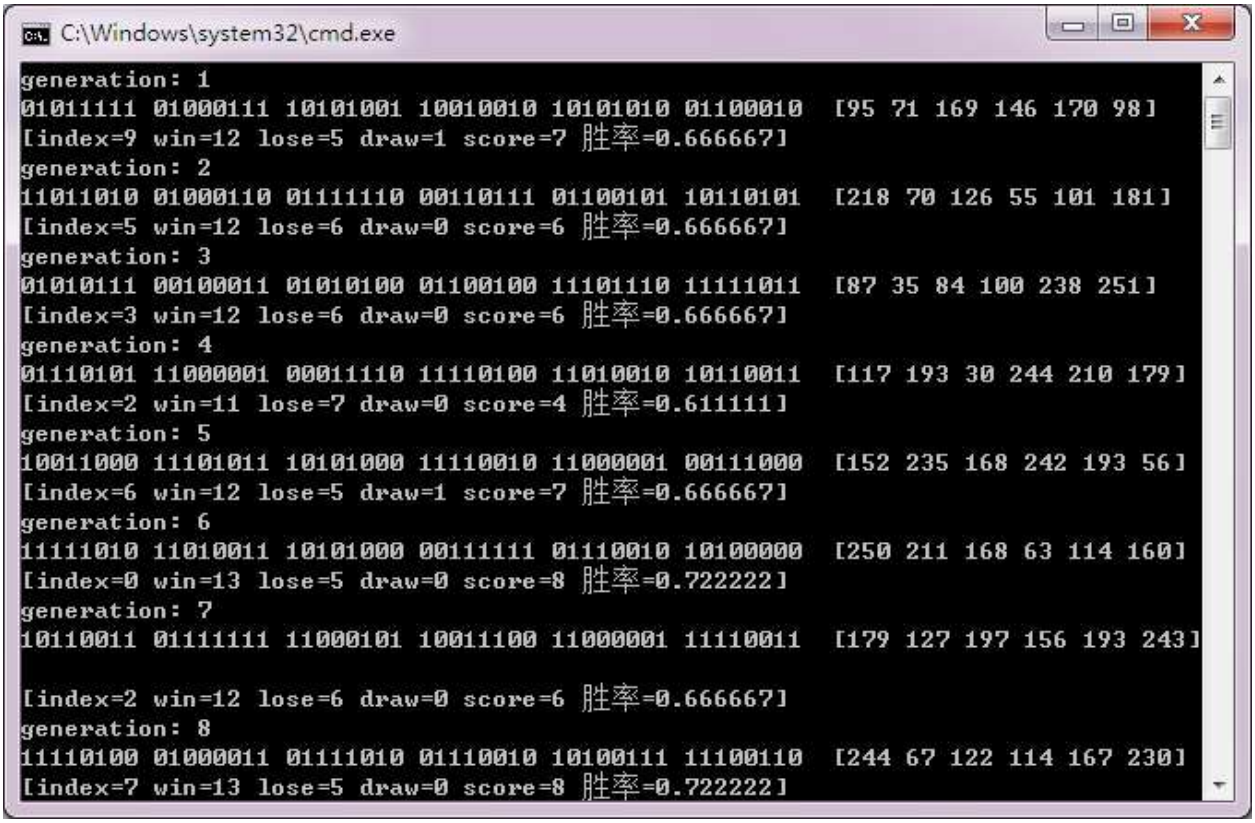
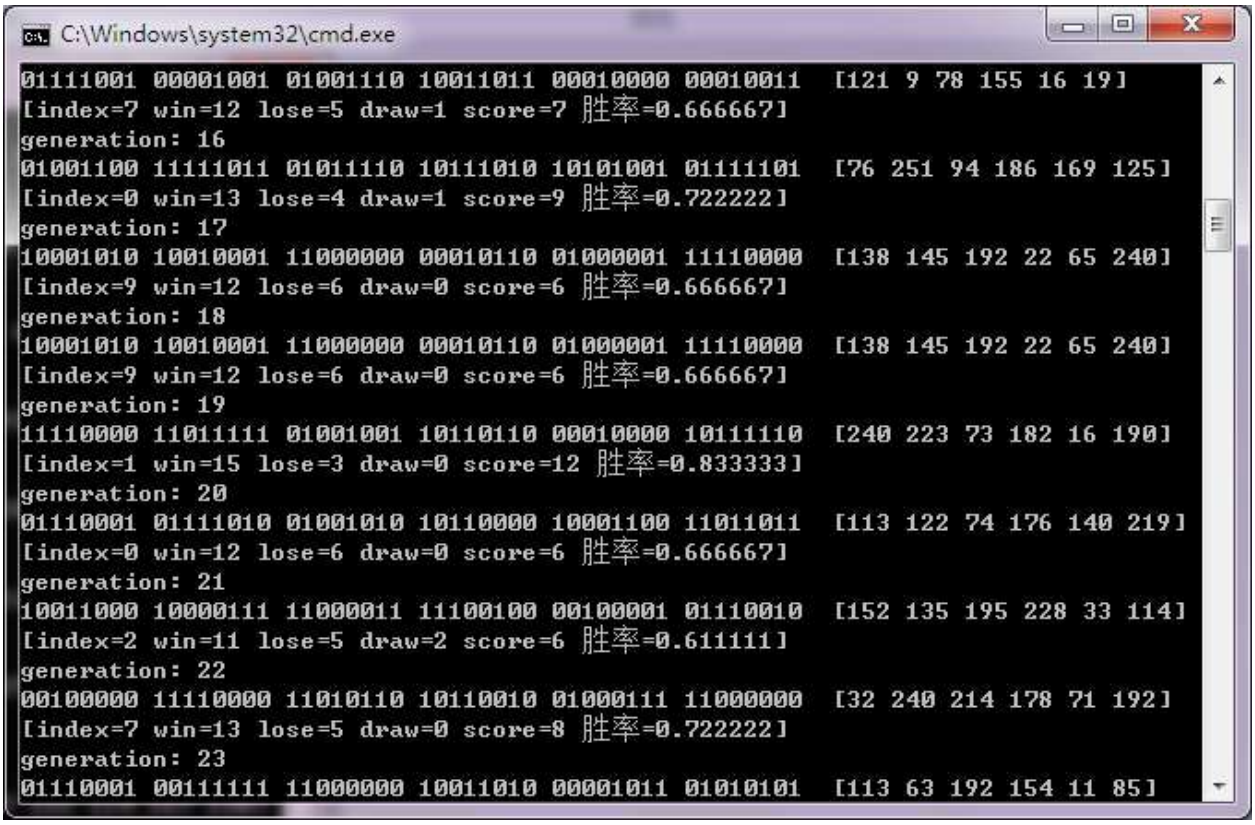
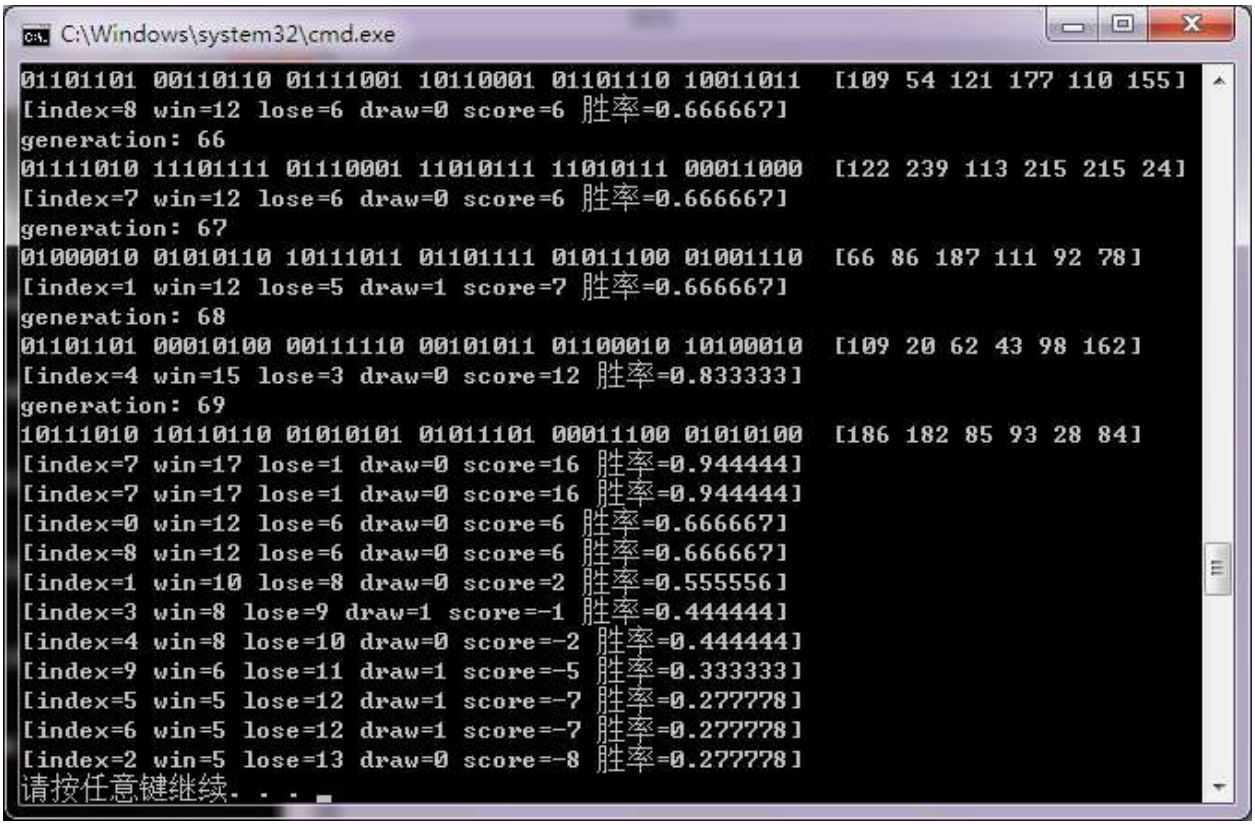


图 5-7　开始遗传



```
C:\Windows\system32\cmd.exe
01111001 00001001 01001110 10011011 00010000 00010011  [121 9 78 155 16 19]
[index=7 win=12 lose=5 draw=1 score=7 胜率=0.666667]
generation: 16
01001100 11111011 01011110 10111010 10101001 01111101  [76 251 94 186 169 125]
[index=0 win=13 lose=4 draw=1 score=9 胜率=0.722222]
generation: 17
10001010 10010001 11000000 00010110 01000001 11110000  [138 145 192 22 65 240]
[index=9 win=12 lose=6 draw=0 score=6 胜率=0.666667]
generation: 18
10001010 10010001 11000000 00010110 01000001 11110000  [138 145 192 22 65 240]
[index=9 win=12 lose=6 draw=0 score=6 胜率=0.666667]
generation: 19
11110000 11011111 01001001 10110110 00010000 10111110  [240 223 73 182 16 190]
[index=1 win=15 lose=3 draw=0 score=12 胜率=0.833333]
generation: 20
01110001 01111010 01001010 10110000 10001100 11011011  [113 122 74 176 140 219]
[index=0 win=12 lose=6 draw=0 score=6 胜率=0.666667]
generation: 21
10011000 10000111 11000011 11100100 00100001 01110010  [152 135 195 228 33 114]
[index=2 win=11 lose=5 draw=2 score=6 胜率=0.611111]
generation: 22
00100000 11110000 11010110 10110010 01000111 11000000  [32 240 214 178 71 192]
[index=7 win=13 lose=5 draw=0 score=8 胜率=0.722222]
generation: 23
01110001 00111111 11000000 10011010 00001011 01010101  [113 63 192 154 11 85]
```

图 5-8　中间过程



```
C:\Windows\system32\cmd.exe
01101101 00110110 01111001 10110001 01101110 10011011  [109 54 121 177 110 155]
[index=8 win=12 lose=6 draw=0 score=6 胜率=0.666667]
generation: 66
01111010 11101111 01110001 11010111 11010111 00011000  [122 239 113 215 215 24]
[index=7 win=12 lose=6 draw=0 score=6 胜率=0.666667]
generation: 67
01000010 01010110 10111011 01101111 01011100 01001110  [66 86 187 111 92 78]
[index=1 win=12 lose=5 draw=1 score=7 胜率=0.666667]
generation: 68
01101101 00010100 00111110 00101011 01100010 10100010  [109 20 62 43 98 162]
[index=4 win=15 lose=3 draw=0 score=12 胜率=0.833333]
generation: 69
10111010 10110110 01010101 01011101 00011100 01010100  [186 182 85 93 28 84]
[index=7 win=17 lose=1 draw=0 score=16 胜率=0.944444]
[index=7 win=17 lose=1 draw=0 score=16 胜率=0.944444]
[index=0 win=12 lose=6 draw=0 score=6 胜率=0.666667]
[index=8 win=12 lose=6 draw=0 score=6 胜率=0.666667]
[index=1 win=10 lose=8 draw=0 score=2 胜率=0.555556]
[index=3 win=8 lose=9 draw=1 score=-1 胜率=0.444444]
[index=4 win=8 lose=10 draw=0 score=-2 胜率=0.444444]
[index=9 win=6 lose=11 draw=1 score=-5 胜率=0.333333]
[index=5 win=5 lose=12 draw=1 score=-7 胜率=0.277778]
[index=6 win=5 lose=12 draw=1 score=-7 胜率=0.277778]
[index=2 win=5 lose=13 draw=0 score=-8 胜率=0.277778]
请按任意键继续. . .
```

图 5-9　终止

5.4 加权系数优化试验结果

5.4.1 结果收敛情况

设置更大规模的种群，和更严格的终止条件重新计算，能获得更好的结果。本文采用 100 个个体组成的种群重新进行了遗传，程序在 2.2GHz 的双核 CPU 上运行了一天多的时间才停止。记录的结果如表 5-1 所示：

表 5-1　遗传算法的收敛结果

代数	染色体（十进制表示）	最高胜率	最低胜率
100	193 99 207 101 83 219	0.63	0.12
200	193 188 248 80 189 231	0.66	0.25
300	207 82 200 54 123 19	0.71	0.15
400	198 236 97 133 210 133	0.72	0.27
500	186 182 85 93 28 84	0.79	0.42
1000	147 142 127 129 26 28	0.65	0.38
2000	174 60 230 200 204 134	0.77	0.41
3000	133 6 51 15 160 133	0.63	0.47
4000	10 85 112 69 165 16	0.59	0.53

5.4.2 优化结果分析

随着遗传算法迭代的次数越来越多，整体上来看，忽略中间的一些不理想的点，种群中的最高胜率呈逐渐减少的趋势，但不会低于 0.5，最低胜率呈逐渐增加的趋势,且不会高于 0.5，且最高与最低二者的差异逐渐减小。较好的符合了种群中个体的随着遗传的进行整体实力慢慢增强的想法。在博弈程序中采用最后得到的较好的染色体即可。

5.5 采用遗传算法的黑白棋程序性能分析

由于遗传算法在前期确定好了较优染色体，因而程序在对弈过程中只需要对每个可走子位置进行直接估值，估值过程为一个线性累加的过程，不需要搜索任何子结点，时间复杂度为 O(1)，效率极高。这也提高了遗传时个体之间比赛的速度，大大减少收敛所需时间。

5.6 应用到黑白棋人机博弈问题的优势和不足

遗传算法是一种基于空间搜索的算法，通过自然选择、遗传、变异等操作以及达尔文的物竞天择、适者生存的理论，模拟自然进化来寻求所求问题的答案。遗传算法的求解过程值一个最优化过程。虽然不能保证所有得到的答案都是最佳答案，但至少能通过一定的方法将误差控制在允许的范围之内。在求解函数优化、组合优化问题中遗传算法往往比传统的数学方法效果更好，将遗传算法应用到黑白棋中的优点是，能方便的借助优胜劣汰的思想来优化估值函数中的加权系数，从而使得人机博弈时，计算机只需对当前局面的每一个可走子的位置进行估值，估值的过程即将多个可能影响我方利益的因素通过上面的加权系数线性累加，最后根据极大极小原理选择走子。整个过程只是在棋盘局面分析可能影响我方利益的因素这部分需要消耗一定的时间，通过简单的线性累加即可完成一步着法的产生操作，完全免去了占用大量时间和资源的博弈树搜索过程，能大大提高程序的效率。另外，由于每一个产生着法的速度很快，在通过遗传算法获取较优的加权系数时，计算适应度函数的速度势必很快，每一代进化就可以在很短的时间内完成，这样同时也提供高了遗传算法实际应用在黑白棋中的效率。

遗传算法在黑白棋中的应用不是十全十美的，虽然在一定程度上黑白棋的着法产生效率能得到很大的提高，但为了获得较优秀的基因，在黑白棋程序开始与人类棋手对弈之前必须通过遗传算法系数优化算子进行规模巨大的计算，往往要经过几千至几万代才能得到较理想的基因。必须要设置合理的适应度函数，和合适有效的终止条件才能保证得到的最优个体确实是比较优秀的个体，这两个函数若不设置好，将直接影响到最终黑白棋的棋力。设计适应度函数和终止条件有很大的难度。本文经过多次相关试验并对结果进行分析，对传统遗传算法进行改进，提出了一种相对有效的方案。

6 系统实现、验证、分析与改进

6.1 系统实现关键技术

6.1.1 系统组成结构

如图 6-1 所示，本文设计的黑白棋人机博弈系统分为人机界面和人机博弈引擎两大部分。

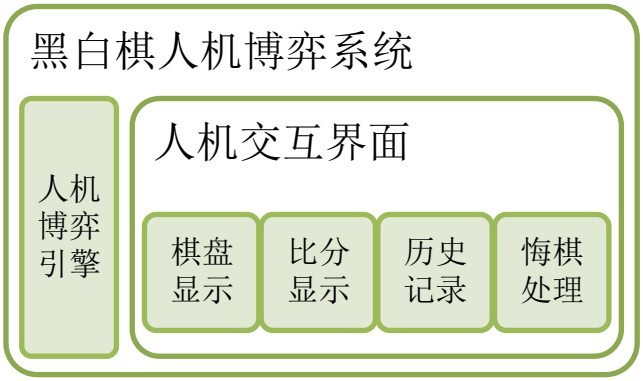


图 6-1　黑白棋人机博弈系统组成结构图

人机界面是一个单独处理用户的输入输出的程序，负责游戏棋盘和比分的显示、下棋历史记录、悔棋等人性化的人机交互接口的处理。博弈引擎作为一个独立的模块，是对黑白棋人工智能算法的封装，对外提供一个调用接口，可供任何遵循该接口的人机界面程序调用。

6.1.2 人机交互界面设计

人机界面是用户与系统的接口，不仅应尽量设计得美观、功能齐全，还应在系统出错时显示出错原因，做出友好提示。除此之外，人机界面还应根据游戏的规则，处理游戏回合的交换和游戏的开局、结束等逻辑，图 6-2 为人机交互界面的截图：

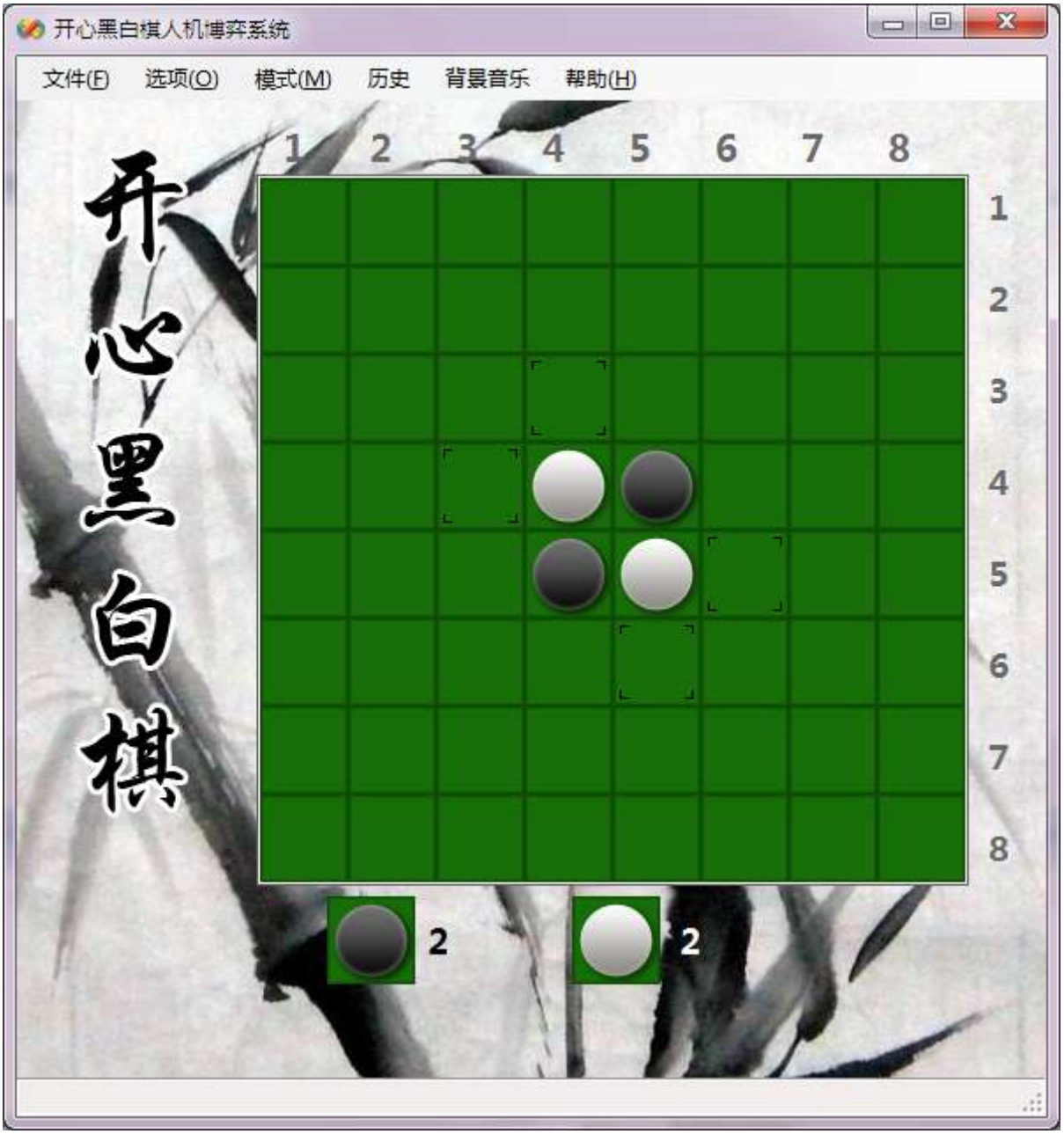


图 6-2　人机交互界面截图

人机界面程序主要具有如下功能：

1. 显示棋盘
2. 提示可以吃子的位置
3. 显示下子记录
4. 显示吃子过程
5. 记录连续轮流欠行的次数
6. 处理游戏回合交换、开局、结局等逻辑

7. 自定义人机对弈或人人对弈
8. 自定义黑子先下或白子先下
9. 历史后退和前进
10. 悔棋，重来
11. 实现计分和显示比分
12. 棋局结束后显示输赢结果

6.1.3 人机博弈引擎设计

人机博弈引擎是一个单独的程序，它为人机交互界面程序提供可能的走法，来实现计算机自动下棋。它封装了黑白棋人工智能算法的处理逻辑，是游戏智能的最终体现。它对外提供一个调用接口，负责接受调用者（通常是人机界面程序）发送来的游戏状态的输入，经过内部的分析和处理，返回一个合适的走法信息或出错信息码给调用者。图 6-3 是人机博弈走法生成过程原理图：

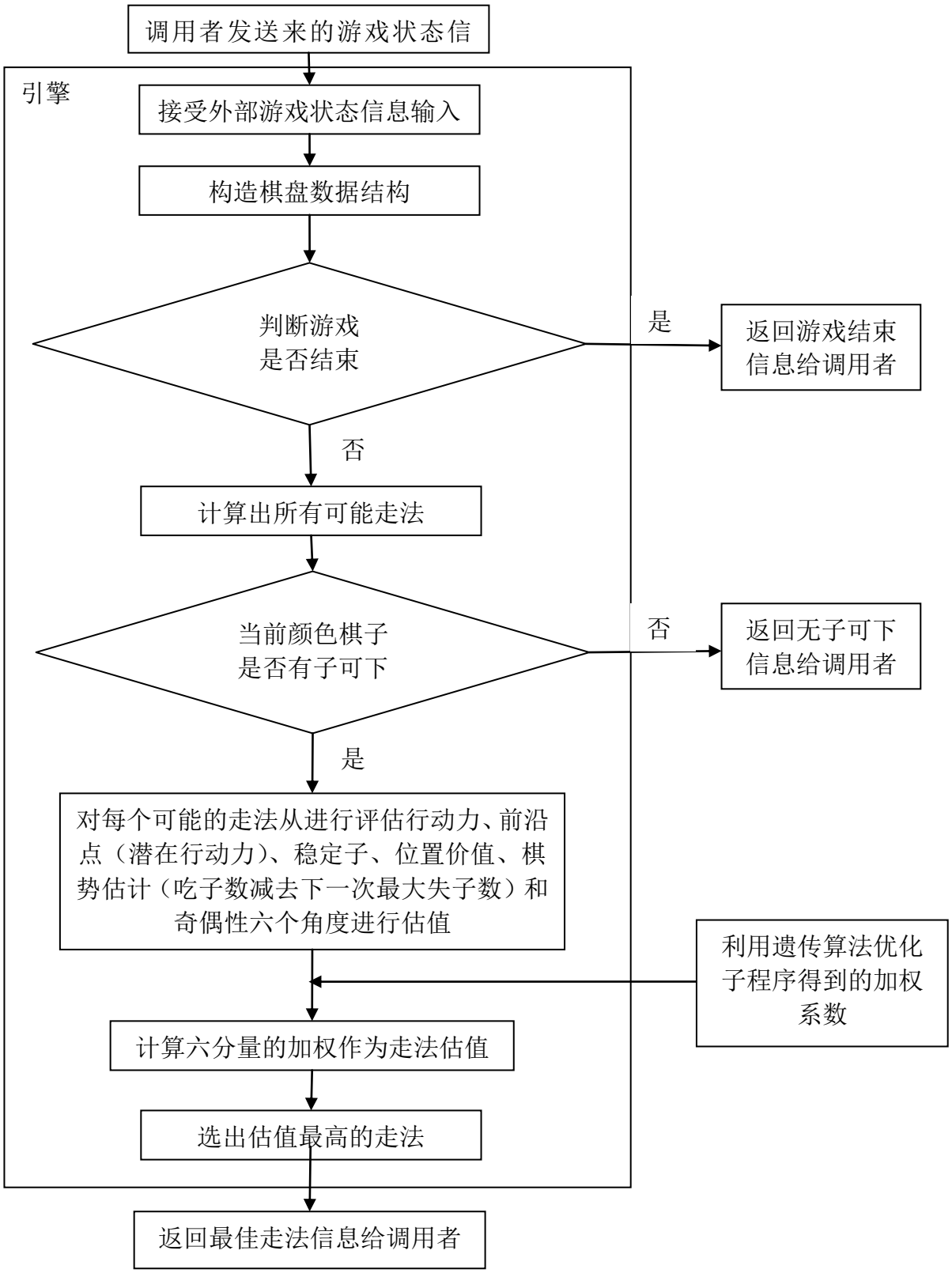


图 6-3　人机博弈引擎走法生成过程原理图

引擎在接受到游戏状态信息并转化为内部数据结构后，首先会分析游戏是否结束，若结束，直接返回游戏结束信息。否则，继续分析当前颜色棋子可走子的位置，若无子可走，说明将发生欠行，直接返回无子可走信息。否则，对各个可走子的位置进行估值并加权取得最好的走法，然后返回走法信息。

若黑白双方连续发生两次欠行，应结束游戏，由于引擎不会去记录两步之间的关联信息，这种情况引擎无法处理，必须有人机交互界面程序处理。

为了最大化提高引擎的处理效率，本程序中的人机博弈引擎使用 C++开发，并大量运用了标准库中的通用容器和算法。编译得到 DLL 动态链接库后可以有 C#制作的人机交互界面调用。

6.1.4 人机博弈引擎调用接口

表 6-1　人机博弈引擎调用接口

名称	说明	示例
游戏状态信息	由一个 64 个字符组成的字符串来表示棋盘上的落子信息，用 0 表示空白棋格，1 表示棋格上落有黑子，2 表示落有白子，最后加上一个代表当前轮到谁下子，同样是 1 表示黑子，2 表示白子	游戏开局时黑子先下可用下面的字符串表示(中间不换行) “0000000000000000 0000000000012000 0002100000000000 00000000000000001”
走法信息	由 2 个字符组成的字符串表示，两个字符是范围在 1 到 8 的数字，代表下子点所处的行和列值	如开局黑子下第 3 行第 5 列用“35”表示
游戏结束信息	用一个数字 1 表示	“1”
无子可走信息	用一个数字 2 表示	“2”

表 6-1 为人机博弈引擎的调用接口描述表，规则较为简单。为了测试引擎的结果是否正确，采用 Visual Basic 设计了一个测试程序，结果如图 6-4、图 6-5 和图 6-6 所示：

图 6-4　引擎测试(黑方无子,游戏已结束)

图 6-5　引擎测试(黑方有子,但无子可下)

图 6-6　引擎测试(黑方有子，且有子可下)

6.2 奇偶性的验证

试验依次使用 3 组相差较大的加权系数确定的具有相同智能程度的机器人互先比赛，从结果可以看出不管智能如何，都是执白后下者胜。大量的实例比赛证明了先下棋者占弱势。实际上，若没有出现欠行，则最后一步总是白棋下子，白棋最后一步下的子永远不可能被吃掉，因而占优势。这个结论应该被考虑到判断另个机器人棋力强弱的因素当中，因为后下者占优势，不能仅评一局棋定胜负分优劣。图 6-7、图 6-8 和图 6-9 为奇偶性验证的试验结果。

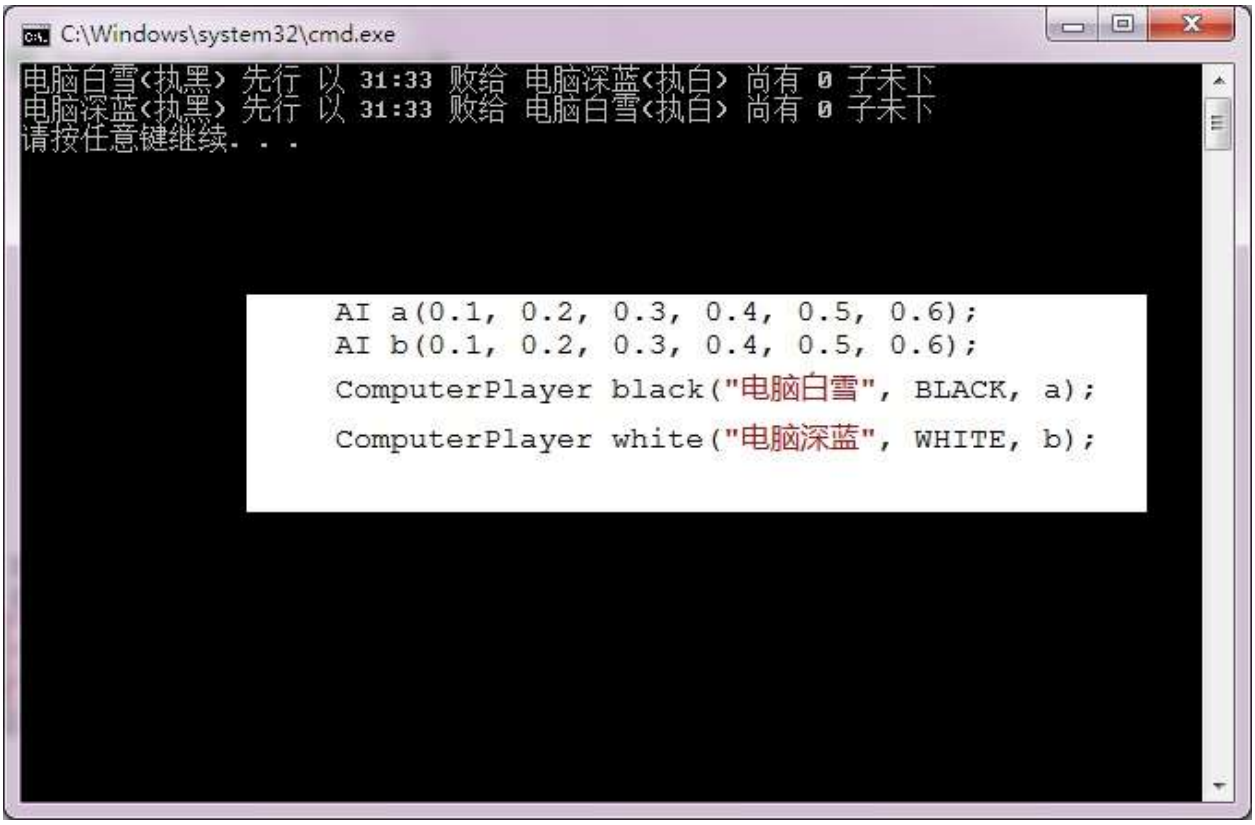


图 6-7　具有相同基因 A 的两个机器人比赛结果

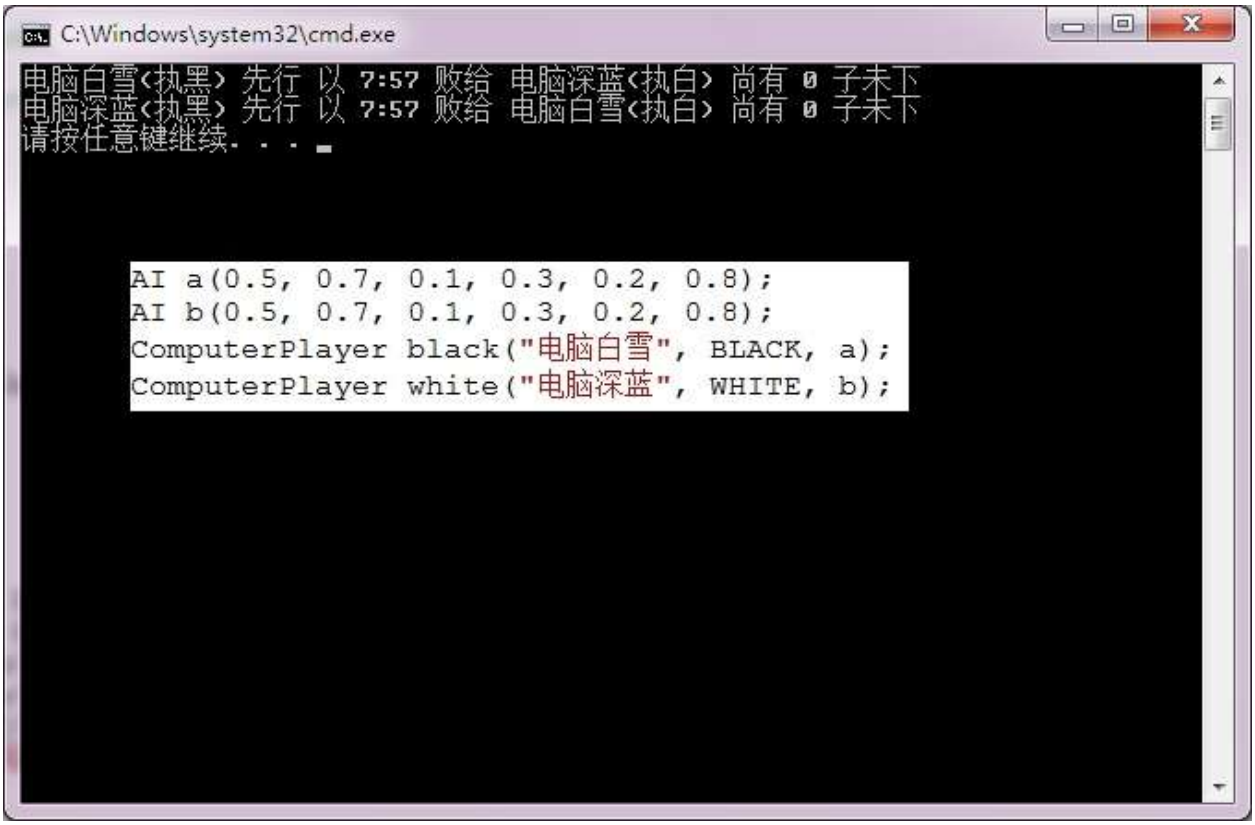


图 6-8　具有相同基因 B 的两个机器人比赛结果

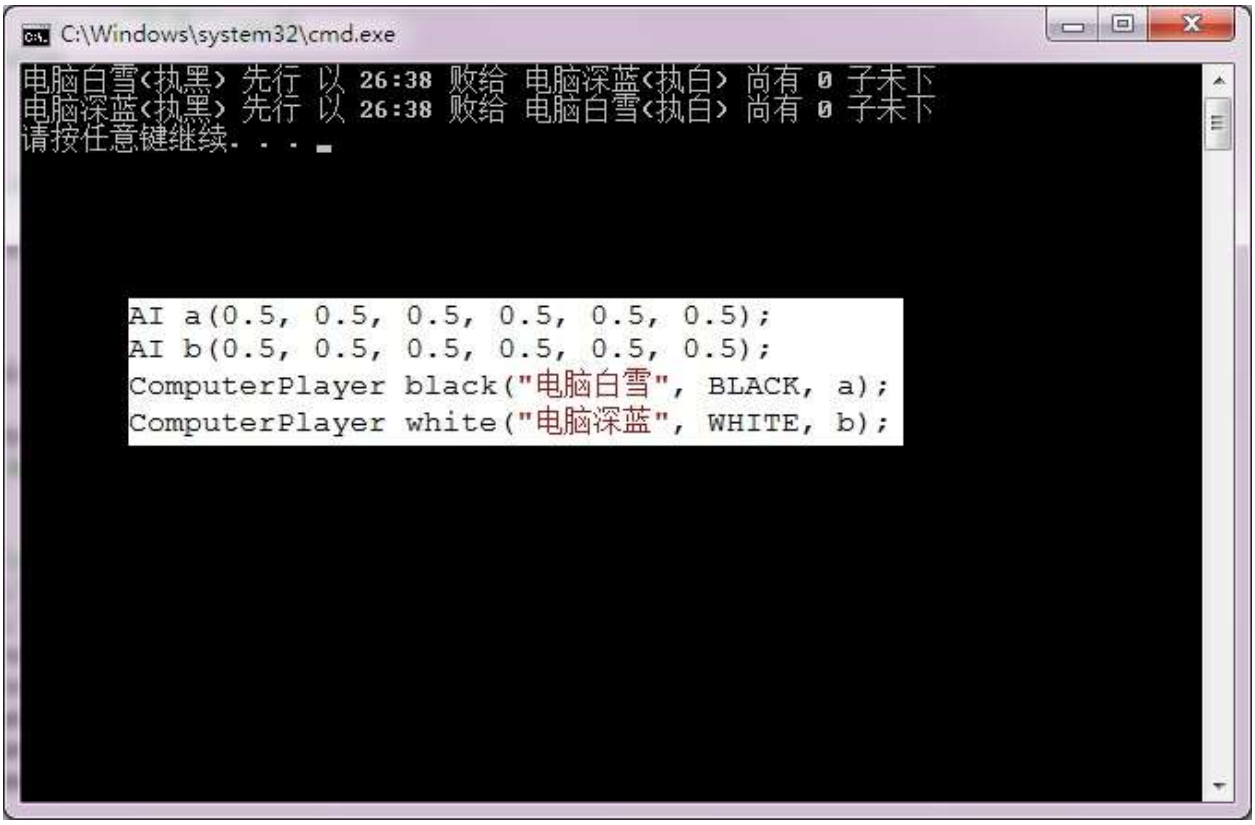


图 6-9　具有相同基因 C 的两个机器人比赛结果

6.3 棋力比较的传递不确定性研究

由于估值函数中包含了行动力、位置价值、奇偶性等因素，其中，行动力是让己方有尽量多的位置下子，而让对手可下子的位置尽量少，对它的计算是直接用下个局面己方可走子位置数减去对方可走子位置数来表示。位置价值和奇偶性也是随着棋局的变化而变化的。当不同棋力的双方下出完全不一样的局面时，计算上述因素得到的值也会有所不同，虽然每个因素对应的权重不变，但最后得到的估值一定是随不同的棋局变化的。可以得出结论，估值的结果是和比赛双方的染色体差异有关的。

假如机器 A 和 B 各自的不同染色体被确定了，它们的棋力便是确定的，A 和 B 某方先行不论下多少局棋，它们每一局棋依次下的每一步都将是一模一样的，是唯一且确定的，不会发生变化，它们对局的结果也是不会发生变化的，永远都是棋力高的一方获胜，A 与 B 的棋力在一定程度上是完全可比的，没有随机性的因素存在。若再有另一个机器人 C，染色体与 A 和 B 都不同，若让 A 与 C 对局，B 与 C 对局，则这两局棋的各个局面从第一着到最后一着一一比较几乎不会每一个局面都完全相同，因而估值时行动力和位置价值的计

算值也不同，估值结果导致各自选择的最佳着子位置也会有差异，最终游戏的胜利是这些差异导致的每一步走棋获得的实际利益偏差综合的结果，相对于比赛双方的棋力具有一定的随机性。假如 A 和 B 的比赛结果是棋力 A 强于 B ，且 B 与 C 的比赛结果是棋力 B 强于 C，仍不能从两场拥有完全不同局面的的比赛中确定 A,B,C 三者的棋力强弱是 A 强于 B 强于，从而错误地推出 A 强于 C。即棋力比较是没有传递性的，A 与 C 的棋力强弱还要具体看 A 和 C 的比赛结果，可能是 A 强于 C，也可能是 C 强于 A，即棋力比较具有传递不确定性。

下面根据前面的估值方法写出棋力比较程序验证了这个观点：

表 6-1　三个不同染色体比赛结果为 A > B, B > C, A < C

	染色体(权值表示)	A(执黑)	B(执黑)	C(执黑)
A(执白)	0.1, 0.2, 0.3, 0.4, 0.5, 0.6	-	A 胜 B 负	C 胜 A 负
B(执白)	0.5, 0.5, 0.5, 0.5, 0.5, 0.5	A 胜 B 负	-	B 胜 C 负
C(执白)	0.6, 0.5, 0.4, 0.3, 0.2, 0.1	A 胜 C 负	B 胜 C 负	-

根据表 6-1 的结果，不能简单的通过基因突变的方法不断的产生新的个体并将比自己强的个体替换自己的方法不断迭代来产生最优个体。

6.4 人机对弈和机机对弈测试

为了测试游戏的棋力和引擎的性能，用制作好的人机界面程序和引擎和已有的游戏引擎进行了一些比赛，另外，用程序和不同的人也进行了一些比赛，按照本文思路设计出的程序表现出为有比较高的棋力，能轻松战胜初级人类玩家，但和国内的较强棋力的程序相比还有一定的差距。

图 6-10 为一个普通棋力的人类玩家和本文设计的程序比赛的比赛结果，人下黑子，计算机下白子，计算机胜了 2 子。

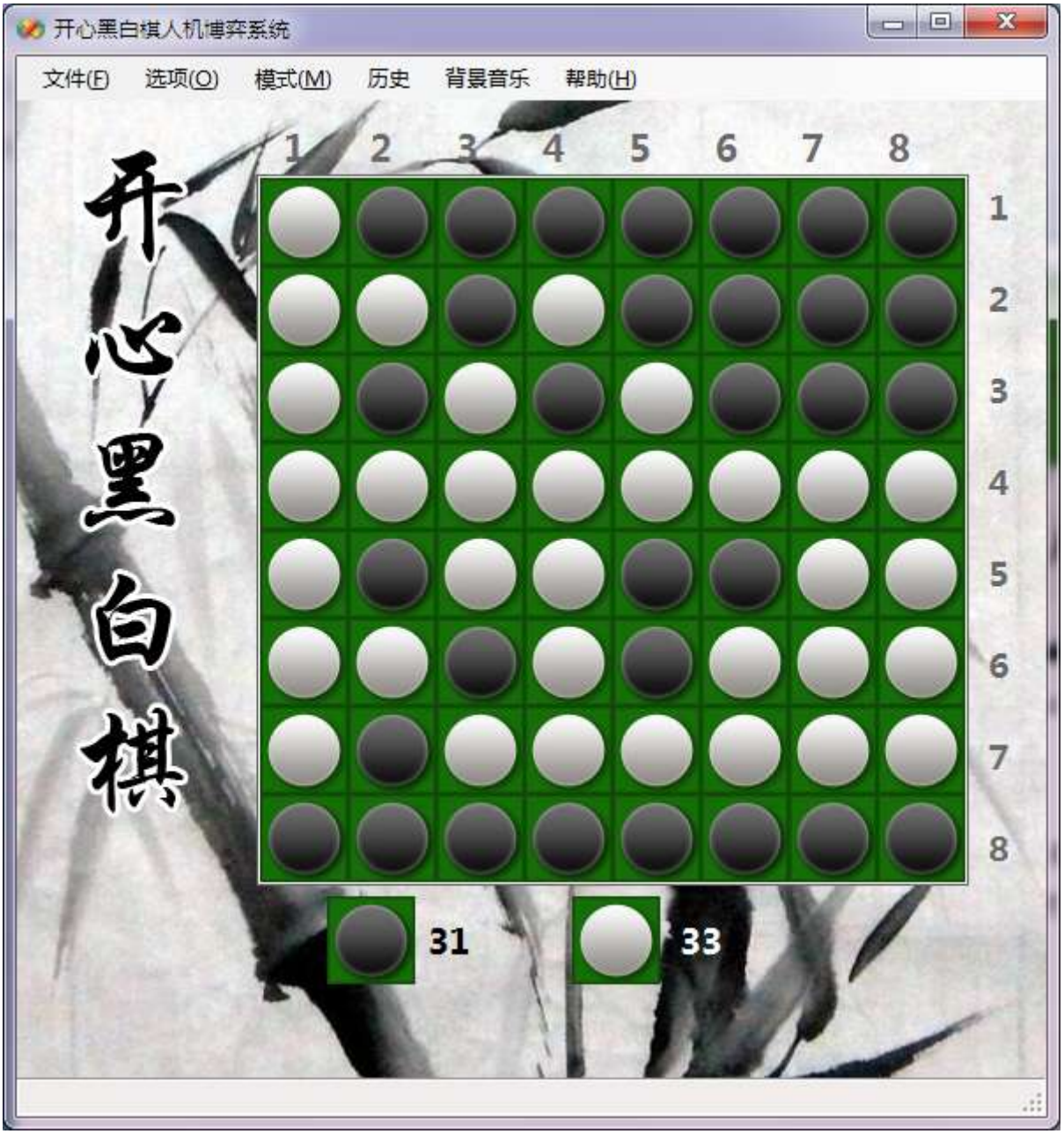


图 6-10 人机对战测试

在 GGS 全球黑白棋对战平台上，本文的程序和机器人 ant 进行了一局比赛，结果本程序战败，但占了一个顶点。图 6-11 为本文的程序和 GGS 上的程序 ant 对局。

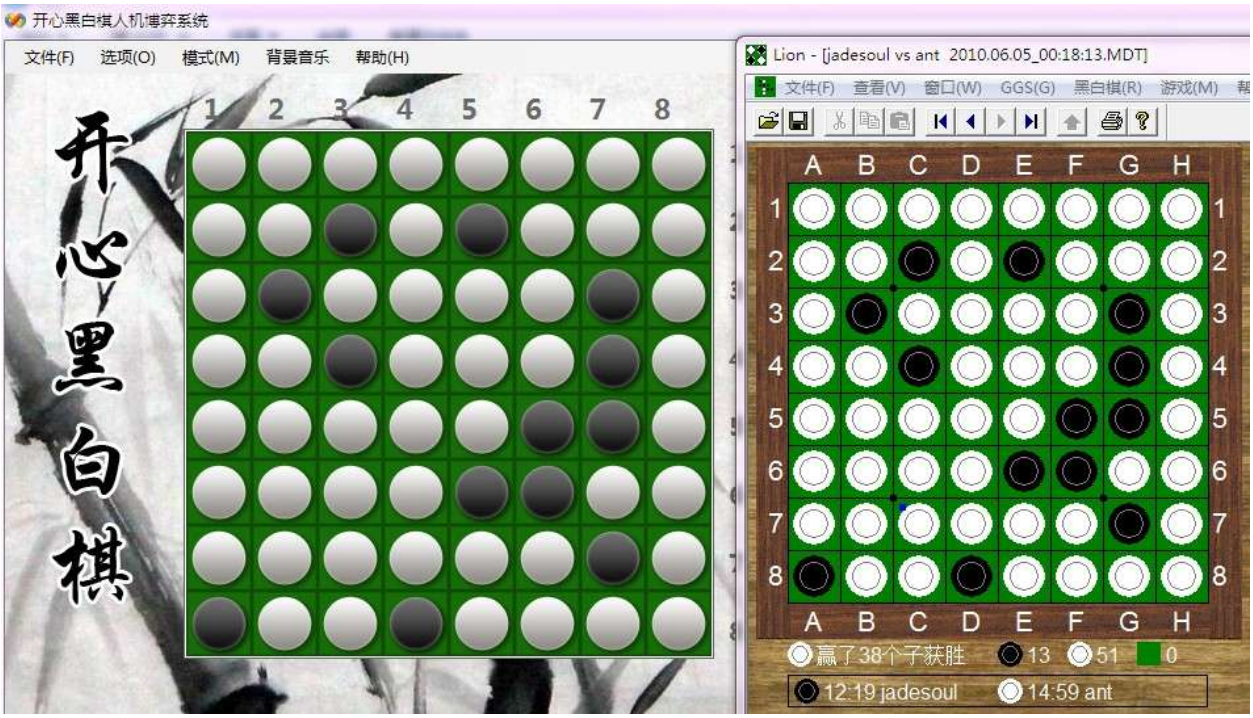


图 6-11 本文的程序和 GGS 上的程序 ant 对局

伤心黑白棋是目前国内黑白棋中棋力最强的程序，本文的程序棋力与它有很大差距，利用本文测试的程序和它对弈，只占据了一个定点，最终战败。图 6-12 是本文的程序与伤心黑白棋的对局中。图 6-13 是比赛结果。

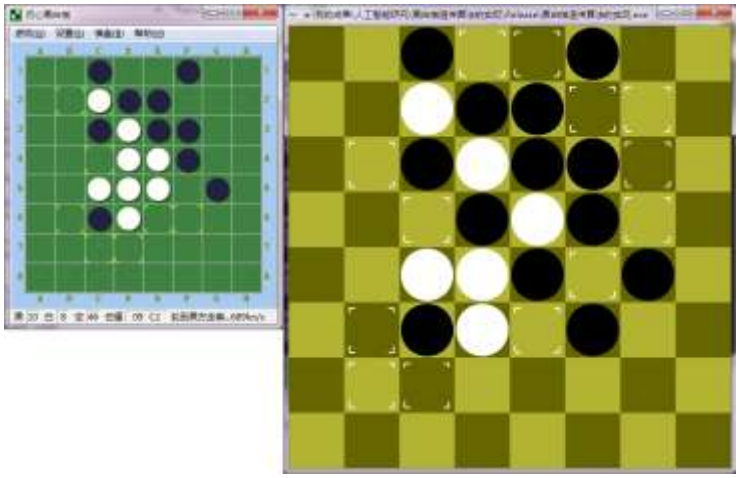


图 6-12 与伤心黑白棋的对局中

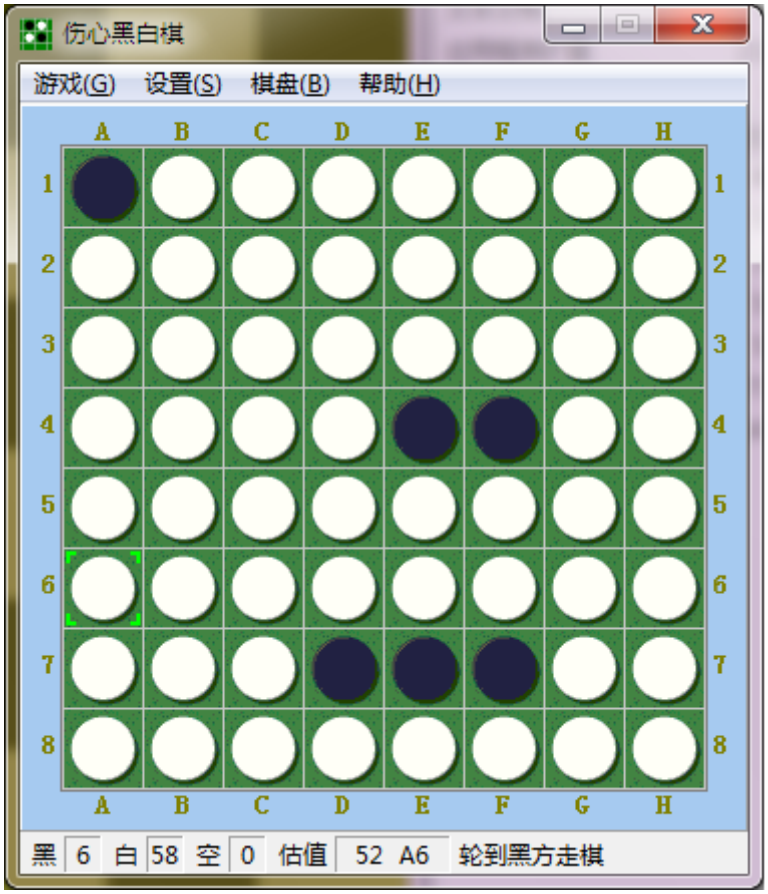


图 6-13 和伤心黑白棋比赛，只占了一个角和若干点

6.5 结果分析与比较

选取不同的较优染色体对应的加权系数得到的不同棋力的程序，和随机的人类棋手各进行了 20 局的棋力测试，记录数据如表 6-2 所示：

表 6-2 遗传不同代数得到的染色体对应的程序和人类棋手比赛结果

代数	染色体	赢	输	平局
100	193 99 207 101 83 219	9	11	0
200	193 188 248 80 189 231	10	9	1
300	207 82 200 54 123 19	12	8	0
400	198 236 97 133 210 133	11	8	1
500	186 182 85 93 28 84	12	6	2
1000	147 142 127 129 26 28	13	7	0
2000	174 60 230 200 204 134	15	5	0

结果表明，经过遗传更多代数得到的染色体对应程序表现出来的棋力更强。

6.6 算法的进一步改进

根据以上验证与分析结构，本文针对黑白棋这一具体问题，从适应度函数和终止条件等角度对传统遗传算法做一些改进，更好发挥遗传算法的作用。另外，从估值的角度出发，采用分阶段的估值方案能大大提高求解效率。

6.6.1 对遗传算法终止条件的改进

为了进一步提高收敛速度，还可以适当改进终止条件，让第一名与倒数第二名或倒数第三名之间的总分差小于最小差别阈值为止。这样可以避免偶然产生了个别特别差的个体的影响，导致虽然其他整体个体整体水平高但仍需要继续遗传，增大遗传代数，同时导致大量优秀个体流失。

更进一步地，可以每次只将与第一名相差不大的连续前 n 名的染色体都保留下来，而其他 N-n 名继续产生变化，这样同时还能持续减少循环赛比赛次数，提高速度。

6.6.2 使用开局库

由于初期的搜索比较简单，只需要让棋子尽量落在中间的 4*4 的区域内即可保证不会产生重大损失，因而游戏初期可以采用开局库，将所有优秀的开局方法加以记录，程序只需要比较是不是当前走法在不在记录中，若在记录中直接按棋谱下子，免去估值的计算。

由于棋盘的对称性，开局棋谱只需要记录从恒定的部分方向下子的结果，其他方向的走法可以通过旋转或对称变换来得到。

6.6.3 分阶段的估值方案

为了进一步提高效率，免去大量重复计算，结合游戏的性质，从分阶段设计的角度出发，可以再对博弈引擎的着法生成部分加以改进。

在前 10 步左右，即游戏的初期，可以采用开局库。

由经验发现，因为游戏中的棋子和局面的变化性较大，前期下子数对终局的影响并不是很大。因而可以在第 10 到第 30 步减少估值因素，例如可以只考虑行动力和位置价值。到了第 30 步开始，一直到 50 步之前，也就是中局，再考虑所有的因素。

到了第 50 步左右的时间，尚未下子的位置数已经不多，程序直接扫描状态空间树的开销不大，能很轻松获得下每个位置最终的输赢情况。在循环赛中，甚至可以直接提前判断胜负以提高效率。

6.6.4 动态遗传算法

之前考虑的遗传算法是在最终程序发布之前提前做大量的比赛来获得较优染色体，一旦获得较优染色体，变将其作为整个程序的加权系数而在程序的运行过程中保持不变，这部分遗传算法是静态的前期处理。但显然，双方轮流下子过程中，局面在不断发生变化，当角色落子之后，角边上的子的位置价值会立刻改变，且取值因敌我双方而不同，在任何一个局面上，各个估值因素对在整个局面确定最佳走子位置影响的强弱程度应该是不同的。可以在下子的过程中动态的改变估值函数所依赖的加权系数。即在确定下子之前，通过基因突变，在当前染色体的基础上产生新的 N-1 个染色体同自身组成一个种群，利用上面的方法进行较少代数的遗传（将阈值调大一些即可），最后获得更适合当前局面的染色体。通过这种动态遗传算法，让拥有当前染色体的个体随着每一步的进行不断的进化。

7 结束语

7.1 总结

经过几个月的努力，终于完成了黑白棋人机博弈系统的设计的以及论文的撰写。设计出了将遗传算法运用到黑白棋游戏的方案，并成功地编写出了程序， 经过测试，程序的棋力达到了一定的水平，通过验证分析，创新改进了遗传算法的应用方式。在老师的悉心指导下，顺利完成了本论文，将工作过程记录了下来。

毕业设计是对个人综合能力的一次大考验，通过这次的毕业设计，感觉自己的无论在理论基础知识上,动手实践方面,还是在心理素质方面都得到了锻炼和提高。在基础知识方面，在平时上课的过程中难免有一些疏漏的地方，在毕业设计的过程中对某些理论知识的理解还不够透彻得以体现出来，通过再次认真复习书本上的相关章节以后，设计进展得更加顺利了。在动手方面，得益于长期坚持练习编程，此次毕业设计代码编写过程进行得十分的顺利。另外，在心理素质方面，我认为毕业设计很能考验一个人的心理素质，在遇到困难的时候要能自己想办法解决，去图书馆找资料、上网去搜索、请教他人，编程是件有趣而繁琐的事，遇到不顺要沉得住气，做什么事情都要耐心。最后，编写出好的程序总是离不开一颗细致的心，编程要细心，一个小小的疏忽往往会叫人白白浪费大量的时间，要勤于动脑，善于思考，好的算法总是深思熟虑的结晶。 要学会帮助他人，勤与与别人交流经验，无论是对方还是自己都能得到提高。总之，在这次毕业设计后自己的理论知识，动手动脑能力，和心理素质都有了一定的提高，经过这次实验，独立完成了毕业设计的制作，获得了许多新的经验，也发现自己在某些地方存在些许欠缺和不足，日后当努力改进。

7.2 展望

本文设计的人机博弈程序虽然能轻松战胜经验不算丰富的人类棋手，但在棋力上比起国内外的优秀程序还有很大的差距，主要是因为程序在以下几个方面还存在着缺点和不足：

1. 在估值过程中，本文的程序只考虑了 6 个因素，而忽略了其他可能不重要的因素，而实际上哪些真正中重要的因素没有合理的论证，可能还没有发现的更重要的因素没有予以考虑。各个因素之间可能会有相互的依赖关系或内部的包含关系，而

这些本程序都没有考虑。

2. 开局库的制作涉及到的工作量巨大，因而在程序中还没有实现。

3. 动态位置价值的计算还不是很完善，数值的确定完全是凭经验进行的，且并没有合理的计算方法。

4. 遗传算法运用到估值的加权系数优化中，虽然节省了向下搜索的开销，但是本质上本文的程序只能向前看一步棋，而传统的基于状态空间搜索的方法往往能向下看 10 到 20 步棋，且具有最后 16 步直接预测结果的完美终局。

5. 在遗传算法中的选择算子的设计过程中，通过循环赛来淘汰个体的机制还不是很完善，没有很好的理论支持，主要原因是棋力比较的不可传递性。

在以后的工作中，将对算法从以上几个方面做进一步的改进。另外，模板估值技术是目前最好最快的解决方案，而自己日后的研究生的学习方向正好和人工智能，机器学习有关，以后将在模板估值方面努力的探索，利用机器学习的方法来提取特征和对大量的棋局做分析和学习，设计出更出色的具有学习能力的黑白棋博弈程序。

参考文献

[1] 杜秀全, 程家兴. 博弈算法在黑白棋中的应用[J]. 计算机技术与发展. 2007, 17(1): 217.

[2] 蔡自兴, 徐光祐. 人工智能及其应用(第三版)[M]. 清华大学出版社, 2004.

[3] 黑白棋天地网站 http://www.soongsky.com/[Z].

[4] Rose Bron. 黑白棋指南[M]. 2005.

[5] 黑白棋百度互动百科http://www.hudong.com/wiki/黑白棋[Z].

[6] 黑白棋百度百科http://baike.baidu.com/view/24242.html[Z].

[7] 黑白棋wiki (http://zh.wikipedia.org/zh-cn/黑白棋)[Z].

[8] 王浩. 四种智能算法的比较研究[J]. 火力与指挥控制. 2008, 33: 71.

[9] 王志水. 基于搜索算法的人工智能在五子棋博弈中的应用研究[J]. 2006.

[10] 李果. 六子棋计算机博弈及其系统的研究与实现[D]. 2007.

[11] 王小春. PC游戏编程[M]. 重庆大学出版社, 2002.

[12] 模板估值技术介绍 (http://blog.sina.com.cn/s/blog_53ebdba00100cpy2.html)[Z].

[13] 马少平, 朱小燕. 人工智能[M]. 清华大学出版社, 2004.

[14] 遗传算法介绍 http://baike.baidu.com/view/45853.htm[Z].

[15]肖其英,王正志.博弈树搜索与静态估值函数.计算机应用研究.1997

[16]黎德玲.中国象棋的机器下棋.北京大学应用数学系硕士毕业论文.2001

[17]徐阳东,刘弘.遗传算法在机器博弈中的创新应用.软件设计开发.2008

[18]徐心和,王骄.中国象棋计算机博弈关键技术分析. 小型微型计算机系统.2006年6月

[19]王永庆.人工智能原理与方法.1998